CodeQuotient

Department of Computer Science & Engineering

LAB MANUAL

22CS025 - Algorithm Design & Implementation-5Sem-2022

Student Name	**SAKSHAM GARG**
Roll Number	
Course Name	**22CS025 - Algorithm Design & Implementation-5Sem-2022**

Faculty Incharge

_____

Table of Contents

Aim: Factorial using recursion

Write a recursive function **factorial** that accepts an integer *n* as a parameter and returns the factorial of *n*, or *n!*.
A factorial of an integer is defined as the product of all integers from 1 through that integer inclusive. For example, the call of **factorial(4)** should return 1 * 2 * 3 * 4, or 24. The factorial of 0 and 1 are defined to be 1.
You may assume that the value passed is non-negative and that its factorial can fit in the range of type int.

Input Format:
The first line of input contains number of testcases , T.
Then T lines follow, which contains an integer,n.

Output Format:
For each testcase print the factorial in new line

Sample Input
2
4
3

Sample Output
24
6

Solution:

```
import java.util.Arrays;
class Result{
  /*
   * Complete the function 'factorial' given below
   * @params
   *  n -> an integer whose factorial is to be calculated
   * @return
   *  The factorial of integer n
   */
  static int helper(int[] dp,int n){
  if (n==1){
      return 1;
    }
    if (dp[n]!=-1){
      return dp[n];
    }
    dp[n]= n*helper(dp,n-1);
    return dp[n];
  }
  static int factorial(int n) {
    // Write your code here
    if(n<1){
      return 1;
    }
    int[] dp=new int[n+1];
    Arrays.fill(dp,-1);
    int res = helper(dp,n);
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    return res;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Sum of all the digits using recursion

Write a recursive function **sumOfDigits** that accepts an integer as a parameter and returns the sum of its digits. For example, calling sumOfDigits(1729) should return 1 + 7 + 2 + 9, which is 19. If the number is negative, return the negation of the value. For example, calling **sumOfDigits**(-1729) should return -19.
Constraints: Do not declare any global variables. Do not use any loops; you must use recursion. Also do not solve this problem using a string. You can declare as many primitive variables like ints as you like. You are allowed to define other "helper" functions if you like; they are subject to these same constraints.

Solution:

```
import java.util.*;
// Other imports go here
// Do NOT change the class name
class Main{
   public static int sumofdigits(int n){
     if (n<=0){
       return 0;
     }
     return n%10+sumofdigits(n/=10);
   }
   public static void main(String[] args)
   {
      // Write your code here
     Scanner sc=new Scanner(System.in);
     int n=sc.nextInt();
     for(int i=0;i<n;i++){
       int m= sc.nextInt();
       int sum=sumofdigits(Math.abs(m));
       if(m<0)
         sum=(-1)*sum;
       System.out.println(sum);
     }
   }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Prime factors using recursion

Write a program that accepts an integer *n* (where n >=2) and print all the prime factors of n using recursion.

Sample Input

24

Sample Output

2
2
2
3

Constraints : Do not declare any global variables. Do not use any loops; you must use recursion. You can declare as many primitive variables like ints as you like. You are allowed to define other "helper" functions if you like; they are subject to these same constraints.

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class Main{
  public static void primefactor(int n,int i){
    if (n<2){
      return ;
    }
     if (n%i == 0){
       System.out.println(i);
       primefactor(n/i,i);
     }else{
       primefactor(n,i+1);
     }
  }
    public static void main(String[] args)
    {
        // Write your code here
      Scanner sc= new Scanner(System.in);
      int n= sc.nextInt();
      primefactor(n,2);
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: power(base, exp)

Write a recursive function **power** that accepts two integers representing a *base* and an *exponent*, and returns the base raised to that exponent. For example, the call to power(3, 4) should return 3^4 i.e. 81. If the exponent passed is negative, then return -1.
Do not use loops or auxiliary data structures; Solve the problem recursively. Also do not use the provided library pow function in your solution.
Expected Time Complexity: O(log(n)); *here n denotes the exponent*
Input Format:
The first line of input conatins an integer T, denoting the number of test cases.
The second line of input contains 2 integers base and exponent seperated by space.
Output Format:
Print the answer when base is raised to the exponent.
Constraints:
-10 <= base <= 10
þÿ-15 <= exponent <= 15
Sample Input
2 // Test Cases
2 3
5 2
Sample Output
8
25

Solution:

```
import java.util.*;
class Result {
   static long power(int base, int exp) {
     if(exp<0){
       return -1;
     }
     if (exp==0){
       return 1;
     }
     if (exp==1){
       return base;
     }
     long hf= power(base,exp/2);
     if (exp%2==0){
       return hf*hf;
     }
     else {
       return base*hf*hf;
     }
   }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Form a new number

Write a recursive function evenDigits that accepts an integer parameter n and returns a new integer containing only the even digits from n, in the same order. If n does not contain any even digits, return 0.
For example, the call of evenDigits(8342116) should return 8426 and the call of evenDigits(35179) should return 0.

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class Main{
    public static int newnum(int n,StringBuilder sb){
      if (n==0){
        return 0;
      }
      if ((n%10)%2==0){
        sb.append(n%10);
      }
      return newnum(n/10,sb);
    }
    public static void main(String[] args)
    {
      // Write your code here
      Scanner sc=new Scanner(System.in);
      int t= sc.nextInt();
      while(t-->0){
        int n = sc.nextInt();
        if (n==0){
          System.out.println(0);
          continue;
        }
        int rev=0;
        while(n!=0){
          rev=(rev*10)+(n%10);
          n/=10;
        }
        // System.out.println(rev);
        StringBuilder sb = new StringBuilder();
        int res= newnum(rev,sb);
        System.out.println(sb.length()==0 ? 0:Integer.parseInt(sb.toString()));
      }
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Binary equivalent using recursion

Write a recursive function **decimalToBinary** that accepts an integer as a parameter and returns an integer whose digits look like that number's representation in binary (base-2). For example, the call of decimalToBinary(43) should return 101011.
Constraints: Do not use a string in your solution. Also do not use any built-in base conversion functions from the system libraries. solve the problem recursively.
Sample Input :
1 // no. of testcases
43
Sample Output :
101011

Solution:

```
class Result{
   static int decimalToBinary(int n){
     int ten=1;
     int res=0;
     while(n!=0){
       int rem = (n & 1)==1 ? 1:0;
       int m=rem * ten;
       res=res+m;
       ten*=10;
       n>>=1;
     }
     return res;
   }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Greatest common divisor using recursion

Write a recursive function **gcd** that accepts two positive non-zero integer parameters *i* and *j* and returns the greatest common divisor of these numbers.

Sample Input

2 // Test Cases
30 18 // i j (testcase 1)
11 17 // i j (testcase 2)þÿ

Sample Output

6
1

Constraints:. Solve the problem recursively.

Solution:

```
class Result
{
  static int gcd(int i, int j)
  {
     while(i!=j){
       if (i>j){
         i-=j;
       }else{
         j-=i;
       }
     }
    return i;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find all pairs with sum K

Given a sorted list of **N** integers, find all distinct pairs of integers in the list with sum equal to a given number **K**, with O(n log n) or O(n) time complexity.
Complete the function below which takes the array and K as parameters and return the number of pairs if any and 0 otherwise.

Input Format:

First line of input will contain a positive integer T = number of test cases. Each test case will contain 2 lines.
First line of each test case contains two integers - N and K.
Next line will contain N numbers separated by space in non-decreasing order.

Constraints:

1 "d T "d 10
1 "d N "d 10^5
-(10^9) "d arr[i], K "d 10^9

Output Format:

For each test case, print number of distinct pairs whose sum will be equal to k. A pair must have two numbers at different indices.
Two pairs are different if at least one of the indices in them is uncommon. Indices - (2,3) and (3,2) are not distinct, but (2,3) and (2,4) are.

Sample Input

```
3  // Test Cases
10 11  // N K (testcase 1)
1 2 3 4 5 6 7 8 9 10
5 10   // N K (testcase 2)
2 4 6 8 10
6 27   // N K (testcase 3)
12 15 20 22 34 36
```

Sample Output

```
5
2
1
```

Solution:

```
class Result {
  static int getPairsCount(int arr[], int n, int k) {
    // Write your code here
      Map<Integer,Integer> mp=new HashMap<>();
      int cnt=0;
      for (int i:arr){
          if(mp.containsKey(k-i)){
              cnt+=mp.get(k-i);
          }
          mp.put(i,mp.getOrDefault(i,0)+1);
      }
      return cnt;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find first occurrence of an integer in a sorted list with duplicates

Given a sorted list of integers, find the position of first occurrence of a given number **K** in the list in O(log n) time.

Input Format:

First line of input will contain a positive integer T = number of test cases.
Each test case will contain the following two lines:
First line will contain two positive integer N = number of elements in list and K.
Second line will contain N space separated integers in increasing order.

Constraints:

1 <= N <= 10^5
-(10^9) <= arr[i], K <= (10^9)

Output Format:

For each test case, print on a single line the index of first occurrence of K in the list on 0-based index. Print -1 if you cannot find K in the list.

Sample Input

```
3  // Test Cases
10 4  // N K (testcase 1)
1 2 4 4 4 4 5 8 9 10
15 7  // N K (testcase 2)
1 2 3 3 5 6 7 7 7 7 7 8 8 8 8
9 1   // N K (testcase 3)
-5 -4 -3 -2 -1 0 0 0 1
```

Sample Output

```
2
6
8
```

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class Main{
    public static int firstOcc(int[] arr,int n,int key){
      int s=0;
      int e=n-1;
      int res=-1;
      while(s<=e){
        int mid= s+(e-s)/2;
        if (arr[mid]==key){
          res=mid;
          e=mid-1;
        }else if (arr[mid]<key){
          s=mid+1;
        }else{
          e=mid-1;
        }
      }
      return res;
    }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    public static void main(String[] args)
    {
       // Write your code here
      Scanner sc= new Scanner(System.in);
      int t=sc.nextInt();
      while(t-->0){
        int n=sc.nextInt();
        int k=sc.nextInt();
        int[] arr= new int[n];
        for(int i=0;i<n;i++){
          arr[i]=sc.nextInt();
        }
        int res= firstOcc(arr,n,k);
        System.out.println(res);
      }
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find count of a number in a sorted list with duplicates

Given a sorted list of integers with duplicates, find the count of a given number **K** in that list in O(log n) time.

Input

First line of input will contain a positive integer T = number of test cases. Each test case will contain 2 lines.
First line of each test case will contain two number N = number of elements in list and K separated by space.
Next line will contain N space separated integers.

Constraints

1 <= N <= 10^5
-(10^9) <= arr[i], K <= (10^9)

Output

For each test case, print on a single line, the count of number K in this list.

Sample Input

```
3  // Test Cases
10 5  // N K (testcase 1)
1 2 2 5 5 5 7 7 7 8
10 1  // N K (testcase 2)
1 1 1 1 1 1 1 2 2 3
20 2  // N K (testcase 3)
1 1 1 1 1 2 2 2 2 2 3 3 3 3 3 4 4 4 4 4
```

Sample Output

```
3
7
5
```

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
class Main{
    public static int firstOcc(int[] arr,int n,int key){
      int s=0;
      int e=n-1;
      int res=-1;
      while(s<=e){
        int mid= s+(e-s)/2;
        if (arr[mid]==key){
          res=mid;
          e=mid-1;
        }else if (arr[mid]<key){
          s=mid+1;
        }else{
          e=mid-1;
        }
      }
      return res;
    }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    public static int lastOcc(int[] arr,int n,int key){
    int s=0;
    int e=n-1;
    int res=-1;
    while(s<=e){
      int mid= s+(e-s)/2;
      if (arr[mid]==key){
        res=mid+1;
        s=mid+1;
      }else if (arr[mid]<key){
        s=mid+1;
      }else{
        e=mid-1;
      }
    }
    return res;
  }
  public static void main(String[] args)
  {
     // Write your code here
    Scanner sc= new Scanner(System.in);
    int t=sc.nextInt();
    while(t-->0){
      int n=sc.nextInt();
      int k=sc.nextInt();
      int[] arr= new int[n];
      for(int i=0;i<n;i++){
        arr[i]=sc.nextInt();
      }
      int res= firstOcc(arr,n,k);
      int res2= lastOcc(arr,n,k);
      System.out.println(res2-res);
    }
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Rotation count of a sorted Array

Given an array of integers, find the minimum number of rotations performed on some sorted array to create this given array.

Input

First line of input will contain a number T = number of test cases. Each test case will contain 2 lines. First line will contain a number N = number of elements in the array (1 <= N <= 50). Next line will contain N space separated integers.

Complete the function **int rotationCount(int array[], int size)** which will receive array and size of the array as input and return how many times the sorted array is rotated. Function should return -1 if the array is not rotated.

Output

Print one line of output for each test case with the minimum number of rotations for given array.

Sample Input:

2
6
15 18 3 4 6 12
6
1 2 3 4 5 6

Sample Output

2
-1

Solution:

```
import java.util.Scanner;
public class Main {
   // Function to find the index of the minimum element (rotation count)
   public static int rotationCount(int[] array, int size) {
      // If the array is not rotated, return -1
      if (array[0] <= array[size - 1]) {
         return -1;
      }
      int low = 0, high = size - 1;
      while (low <= high) {
         // If the subarray is already sorted, return the lowest index
         int mid = low + (high - low) / 2;
         int next = (mid + 1) % size;
         int prev = (mid + size - 1) % size;
         // Check if the mid element is the minimum
         if (array[mid] <= array[next] && array[mid] <= array[prev]) {
            return mid;
         } else if (array[mid] <= array[high]) {
            high = mid - 1; // Search in the left half
         } else if (array[mid] >= array[low]) {
            low = mid + 1; // Search in the right half
         }
      }
      return -1; // Just a fallback, this should never be reached
   }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt(); // Number of test cases
        for (int t = 0; t < T; t++) {
            int N = sc.nextInt(); // Number of elements in the array
            int[] array = new int[N];
            for (int i = 0; i < N; i++) {
                array[i] = sc.nextInt();
            }
            int result = rotationCount(array, N);
            System.out.println(result);
        }
        sc.close();
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Search element in a rotated sorted array

Given an array of **n** integers which is sorted but rotated by some number of times after sorting, and a integer **k**. Search the element k in this sorted rotated array efficiently. For example, suppose an array sorted in ascending order is rotated at some pivot unknown to you beforehand. (i.e., 0 1 2 4 5 6 7 might become 4 5 6 7 0 1 2).
Assume there are no duplicate elements in the array.
Expected Time Complexity: O(log(N))
Expected Space Complexity: O(1)
Input Format
First line of input will contain a number T = number of test cases. Each test case will contain 3 lines. The first line will contain an integer k to be searched. Second line will contain a number n = number of elements in the array. Next line will contain N space separated integers.
Complete the function int searchRotatedSortedArray() to search a value target in array a of size given, and if target found in the array return its index, otherwise return -1.
You may assume no duplicate exists in the array.
Output Format
Print the index of k in given array for each test case in new line if found and print -1 if k is not present in given array.
Constraints
1 <= T <= 10
-1000 <= k <= 1000
1 <= n <= 10^5
-1000 <= arr[i] <= 1000
Sample Input
2 // Test Cases
3 // k (testcase 1)
6 // n
15 18 2 3 6 12 // arr[]
7 // k (testcase 2)
7 // n
4 5 8 9 1 2 3 // arr[]
Sample Output
3
-1

Solution:

```
class Result {
  static int pivot(int[] arr){
    int n=arr.length;
    int s=0;
    int e=n-1;
    while(s<e){
      int mid= s+(e-s)/2;
      if (arr[0]<=arr[mid]){
        s=mid+1;
      }else{
      e=mid;
      }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    }
    return s;
  }
  static int Bs(int[] arr,int key,int s,int e){
    while(s<=e){
      int mid=s+(e-s)/2;
      if (arr[mid]==key){
        return mid;
      }
      else if(arr[mid]>key) {
        e=mid-1;
      }else{
        s=mid+1;
      }
    }
    return -1;
  }
  static int searchRotatedSortedArray(int arr[], int k) {
    // Write your code here
    int pivt= pivot(arr);
    //System.out.println(pivt);
    int n=arr.length;
    if (k>=arr[pivt] && k<=arr[n-1]){
      return Bs(arr,k,pivt,n-1);
    }else{
      return Bs(arr,k,0,pivt);
    }
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Reverse the order of words of a string

Given a string of words, reverse the order of words in the string individually, not the whole string.
Complete the function **revWordsOrder(str)** that accepts a string as parameter and reverses the order of words of string.
Input Format:
The first line of input contains an integer T denoting the no of test cases . Then T test cases follow. Each test case contains the string str.
Output Format:
For each test case, print the resultant string in new lines.
Sample Input
2
Code Quotient Loves Code
Hello Coders
Sample Output
Code Loves Quotient Code
Coders Hello

Solution:

```
import java.util.*;
class Result {
  static String revWordsOrder(String str) {
    // Write your code here
    String[] strarr= str.split(" ");
    String ansstr=new String();
    for (int i=strarr.length-1;i>=0;i--){
      ansstr+=strarr[i]+" ";
    }
    return ansstr.trim();
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: String is subsequence or not

Given two strings, find whether second string is subsequence of first string. A subsequence of a string is defined as a string that can be obtained by deleting any number of characters from the original string.
Complete the function **strSubsequence(str1, str2)** that accepts two strings as parameters and print **YES** if str2 is a subsequence of str1 and **NO** otherwise.

Input Format

The first line of input contains an integer T denoting the no of test cases . Then T test cases follow.
Each test case contains a single line of input which contains two strings str1, str2 seperated by a space.

Output Format

For each test case, print YES or NO in new lines.

Constraints

1 <= T <= 10
Given strings consist of uppercase and lowercase English letters.

Sample Input

3
CodeQuotient CQ
CodeQuotient QC
Hello Hi

Sample Output

YES
NO
NO

Solution:

```
class Result{
  // Return true if the str2 is a subsequence of str1, else return false
  static boolean strSubsequence(String str1, String str2) {
    // Write your code here
    int m=str1.length();
    int n=str2.length();
    int j=0;
    for (int i=0;i<m && j<n;i++){
      if(str1.charAt(i)==str2.charAt(j)){
        j++;
      }
    }
    if (j==n){
      return true;
    }else{
      return false;
    }
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Technical Issue With The Keyboard

Aman likes to play with strings, so his friend Riya decided to send him a mail, containing a sorted string with unique characters. But, due to some technical issue with her keyboard, whenever she presses a key, the corresponding character gets printed multiple times. Now Riya needs your help for removing all the unwanted characters from her string. For example, If **aabbbccdef** is a sorted string generated by Riya, then you should output the following string as answer **abcdef**.
Complete the function **getDesiredString(str)** that will take the string as parameter and modify it as specified.
Input Format:
The first line of input contains an integer T denoting the no of test cases . Then T test cases follow. Each test case contains a string str.
Output Format:
For each test case, print the new string in new lines.
Constraints:
1 <= T <= 10
1 <= length of string <= 10^5
Each string will contain only lower-case English letters.þÿ
Sample Input
2
aabbccdef
abddddddd
Sample Output
abcdef
abd

Solution:

```
class Result {
  // Return the updated string
  static String getDesiredString(String str) {
    // Write your code here
    int n=str.length();
    String ans=new String();
    for (int i=0;i<n;i++){
      ans+=str.charAt(i);
      while(i<n-1 && str.charAt(i)==str.charAt(i+1)){
        i++;
      }
    }
    return ans;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Implement atoi and itoa functions

Implement the below functions with recursion from string library as your own functions.
1. itoa() function converts int data type to string data type.
2. atoi() function converts string data type to int data type.

Input Format:

The first line of input contains an integer T denoting the no of test cases . Then T test cases follow. Each test case contains a string str and a number num.
Function void itoa(int num, char* result) will take the number as parameter and place the string from this number in array result.
Function int atoi(char *str) will take the string as parameter and return the string as an integer value.

Output Format:

For each test case, print the output of itoa() and atoi() in new lines.

þÿSample Input

1
"100" 135

Sample Output

100 "135"

Solution:

```
class Result {
  static String itoa(int num) {
    // Write your code here
    String ans=Integer.toString(num);
    return ans;
  }
  static int atoi(String str) {
    // Write your code here
    int ans= Integer.parseInt(str);
    return ans;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Implement strcat function

Implement the strcat() function from string library as your own function. The function will take two strings as arguments and concatenate the second string at the end of first string.

Input Format:

The first line of input contains an integer T denoting the no of test cases .
Then T test cases follow. Each test case contains two strings str2 and str1.
Function strcatCode(str1,str2) will take two parameters and concatenate the str2 string at end of str1.

Output Format:

For each test case, print the concatenated string in new lines.

Sample Input

1
Code Quotient

Sample Output

CodeQuotient

Solution:

```
static String strcatCode(String a, String b) {
  // Write your code here
  return a+b;
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Count words

Write a function **countWords()** to count the numbers of words in a string.
A word is defined as text separated by space(' ') or multiple spaces.
The function will receive a string as input and return the numbers of words in this string.
Input Format
A single line of input which consists of the string whose words is to be counted
Output Format
Print the count the numbers of words in a string
Sample Input
Codequotient get better at coding
Sample Output
5

Solution:

```
class Result {
  static int countWords(String str) {
  // Write your code here
    return (str.length())==0? 0 : str.split("[ ]+").length;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Spell the number

Given a non-negative integer between 0 and 9,99,999 print the english phrase corresponding to it i.e. 999999 is “nine lakhs ninety nine thousand nine hundred ninety nine”.

þÿInput Format:

The first line of input contains an integer T denoting the no of test cases . Then T test cases follow. Each test case contains one integer number.

Function void intToWord(int num) will take the number as parameter and print the english phrase for it in small letters separated by space.

Output Format:

For each test case, print the english phrase in new lines.

Sample Input

2 // No. of test cases
31
12345

Sample Output

thirty one
twelve thousand three hundred forty five

Solution:

```
import java.util.Scanner;
class Result {
     // Arrays to hold words for digits and tens
     private static final String[] LESS_THAN_TWENTY = {"", "one", "two", "three", "four",
"five", "six", "seven", "eight", "nine", "ten", "eleven", "twelve", "thirteen", "fourteen",
"fifteen", "sixteen", "seventeen", "eighteen", "nineteen"};
     private static final String[] TENS = {"", "", "twenty", "thirty", "forty", "fifty", "sixty",
"seventy", "eighty", "ninety"};
     private static final String[] THOUSANDS = {"", "thousand", "lakh"};
    static void intToWord(int n) {
      // Write your code here
      System.out.print(convert(n));
     }
     // Function to convert number to words
      static String convert(int num) {
       if (num == 0) {
           return "zero";
       }
       StringBuilder result = new StringBuilder();
       if (num >= 100000) {
         result.append(LESS_THAN_TWENTY[num / 100000]).append(" lakhs ");
         num %= 100000;
       }
      if (num >= 1000) {
        result.append(convertLessThanThousand(num / 1000)).append(" thousand ");
        num %= 1000;
      } if (num > 0) {
        result.append(convertLessThanThousand(num));
      }
       return result.toString().trim();
   }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    // Helper function to convert numbers less than 1000
    static String convertLessThanThousand(int num) {
        if (num < 20) {
            return LESS_THAN_TWENTY[num];
        } else if (num < 100) {
            return TENS[num / 10] + (num % 10 > 0 ? " " + LESS_THAN_TWENTY[num %
10] : "");
        } else {
            return LESS_THAN_TWENTY[num / 100] + " hundred" + (num % 100 > 0 ? " " +
convertLessThanThousand(num % 100) : "");
        }
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Check if strings are rotations or not

Given two strings, find whether both are rotations of each other or not. For example,
Given str1 = abacd and str2 = acdab, we can get str1 by rotating str2 and,
Given str1 = coder and str2 = cored, we can not get str1 by rotating str2.

þÿInput Format

The first line of input contains an integer T denoting the no of test cases. Then T test cases follow. Each test case contains two strings str1 and str2.

Output Format

For each test case, print YES or NO in new lines.

Sample Input

```
2  // Test Cases
abacd  // testcase 1
acdab
coder  // testcase 2
cored
```

Sample Output

```
YES
NO
```

Solution:

```
class Result {
  // return 1 for YES and 0 for NO.
  static int isRotation(String str1, String str2) {
    // Write your code here
    if (str1.length()!=str2.length())
      return 0;
    String s3=str1+str1;
    if (s3.indexOf(str2)!=-1){
      return 1;
    }else{
      return 0;
    }
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: First Unique Character

Given a string that contains only lowercase English letters, find the first non-repeating character in it and return its index. If it doesn't exist, then return -1.

Input Format:
First line contains a string.

Output Format:
Print the index of the first non-repeating character in the given string. If it doesn't exist, then print -1.

Constraints:
'a' <= str[i] <= 'z'
1 <= length of str <= 100000

Sample Input 1
codequotientchamp

Sample Output 1
2

Explanation 1
'd' is the first non-repeating character in the given string, therefore the output is its index i.e. 2.

Sample Input 2
silentlisten

Sample Output 2
-1

Explanation 2
All the characters in the given string are repeating, therefore the output is -1.

Solution:

```
import java.util.*;
class Result {
  static int firstUniqueChar(String str) {
    // Write your code here
      int n=str.length();
      Map<Character,Integer> mp=new HashMap<>();
      for (char ch:str.toCharArray()){
          if (mp.containsKey(ch)){
              mp.put(ch,mp.get(ch)+1);
          }
          else{
              mp.put(ch,1);
          }
      }
      for (int i=0;i<n;i++){
          if (mp.get(str.charAt(i))==1){
              return i;
          }
      }
      return -1;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find all pairs with K difference

Given an array of **N** distinct integers, find all the pairs of integers in it, with the difference equals to a given number **K**.
Complete the function in the editor, which takes the array and K as parameters and return the number of pairs with the difference K.

Input Format:

First line of input will contain a positive integer T = number of test cases. Each test case will contain 2 lines.
First line of each test case contains two positive integers, N and K.
Next line will contain N distinct numbers separated by space.

Output Format:

For each test case, print number of pairs whose difference will be equal to k.

Constraints:

1 "d T "d 10
1 "d N "d 10^5
1 "d K "d 10^8
-(10^7) "d arr[i] "d 10^7

Sample Input

3 // Test cases
10 7 // N K (testcase 1)
1 2 3 4 5 6 7 8 9 10
5 4 // N K (testcase 2)
4 2 3 1 10
6 27 // N K (testcase 3)
10 15 38 22 11 36

Sample Output

3
0
1

Solution:

```
class Result {
  static int getPairsCount(int arr[], int n, int k) {
    // Write your code here
      Set<Integer> set=new HashSet<>();
      int cnt=0;
      for (int i:arr){
          set.add(i);
      }
      for (int i:arr){
          if(set.contains(i+k)){
              cnt++;
          }
      }
      return cnt;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find out the winner

Given an array of strings, find out the string which occurs maximum number of times. If two strings occurs maximum times, return the alphabetically later string. For example,
if array is ["Amit", "Girdhar", "Amit", "Girdhar", "Girdhar", "Amit", "Amit"] then return "Amit", and
if array is ["Amit", "Girdhar", "Amit", "Girdhar", "Girdhar", "Amit"] then return "Girdhar" as both occurs 3 times but Girdhar comes after Amit in alphabetical order.
So, write a function which accepts a string array as input and return the output string.

Input Format

The first line contains an integer n, the number of names in string array.
Each the n subsequent lines contains a string describing array[i] where 0 "d i < n.

Output Format

Print the output string

Constraints

1 "d n "d 10^5
1 "d length of string "d 64
string will contain only lowercase english alphabets i.e. from 'a' to 'z'

Sample Input

6
Amit
Girdhar
Amit
Girdhar
Girdhar
Amit

Sample Output

Girdhar

Solution:

```
class Result{
  /*
   * Complete the 'inspectStrings' function below.
   * @params
   *   words -> A string array, which contains the strings
   *
   * @return
   *   A String, which occurs the maximum numbers of times
   */
  static String inspectStrings(String[] words) {
    // Write your code here
    int n= words.length;
      Map<String,Integer> mp= new HashMap<>();
      for (String s:words){
          if (mp.containsKey(s)){
              mp.put(s,mp.get(s)+1);
          }else{
              mp.put(s,1);
          }
      }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        String res="";
        int mx=-1;
        for (Map.Entry<String,Integer> et:mp.entrySet()){
            //System.out.println(et.getKey()+" "+et.getValue());
            if (mx < et.getValue() || et.getValue()== mx && et.getKey().compareTo(res)>0 ){
                mx=et.getValue();
                res=et.getKey();
            }
        }
        return res;
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Unique characters or not

Given a string, you need to test the characters for their uniqueness. If all the characters occur at most 1 time in the string, then print “YES”, otherwise if some character occurs at least twice in the string print “NO”.

Input Format:

The first line of input contains an integer T denoting the no of test cases . Then T test cases follow. Each test case contains the string str.

Function void isUniqueChars(char *str) will take the string as parameter and print YES or NO according to the occurence of characters in it.

Output Format:

For each test case, print YES or NO in new lines.

Constraints:

1 <= T <= 10

Given string can contain any valid ASCII character.

Sample Input

2
CodeQuotient
Coding

Sample Output

NO
YES

Solution:

```
import java.util.*;
class Result{
  // Return true if string contains all unique characters, else return false
  static boolean isUniqueChars(String str){
    // Write your code here
      Map<Character,Integer> mp= new HashMap<>();
      for (char c:str.toCharArray()){
          if (mp.containsKey(c)){
              mp.put(c,mp.get(c)+1);
          }else{
              mp.put(c,1);
          }
      }
      for (Map.Entry<Character,Integer> et:mp.entrySet()){
          if (et.getValue()>1) return false;
      }
      return true;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Count frequency of each character

Given a string that contains only lowercase characters. Write a program to print all the distinct characters along with their frequency in the order of their occurrence. For example if the string is “helloworld”, then you should print **h1 e1 l3 o2 w1 r1 d1**
Complete the given function **countFrequency()** and print the frequency of each character as per the given statement.

Input Format

First line contains a string with lowercase characters.

Constraints

'a' <= str[i] <= 'z'
1 <= length of str <= 100000

Output Format

Print all the distinct characters along with their frequency followed by a space. And the characters must be printed in the order of their occurrence.

Sample Input

codequotient

Sample Output

c1 o2 d1 e2 q1 u1 t2 i1 n1

Solution:

```
import java.util.HashMap;
import java.util.Map;
class Result {
    static void countFrequency(String str) {
        // Create a map to store frequency of characters
        Map<Character, Integer> freq = new HashMap<>();
        // Count the frequency of each character in the string
        for (char c : str.toCharArray()) {
            freq.put(c, freq.getOrDefault(c, 0) + 1);
        }
        // Iterate over the string to print each character with its frequency
        // We keep track of already printed characters to avoid duplicates
        Set<Character> printed = new HashSet<>();
        for (char c : str.toCharArray()) {
            if (!printed.contains(c)) {
                System.out.print(c + "" + freq.get(c) + " ");
                printed.add(c);
            }
        }
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Kth distinct element

Given an array of positive integers, and an integer **K**, your task is to find the **K**th distinct element present in the array. If there are fewer than K distinct elements, then return 0 as the answer.
A distinct element is an element which is present only once in an array.
Note: The elements are considered in the same order in which they appear in the array from left to right.
Example: arr[] = {6, 11, 4, 11, 9, 4}, K = 2
The only distinct elements in the array are 6 and 9.
6 appears first so it the 1st distinct element, and 9 appears second so it the 2st distinct element in the array. Hence, for K=2 the answer is **9**.
Input Format:
First line of input contains T = number of test cases.
Each test case contains three lines:
First Line will contain an integer N, denoting the size of the array.
Second line contains N integers separated by space, denoting the array elements.
Third line contains an integer representing K.
Constraints:
1 <= T <= 10
1 <= N <= 10^5
1 <= arr[i] <= 10^5
1 <= K <= N
Output Format:
Print the Kth distinct element present in the array.
Sample Input 1

```
3  // Test Cases
6          // N (testcase 1)
6 11 4 11 9 4 // arr[]
2          // Kþÿþÿ
5          // N (testcase 2)
7 6 7 3 6     // arr[]
1          // K
6          // N (testcase 3)
8 5 3 5 5 5   // arr[]
4          // K
```

Sample Output

```
9
3
0
```

Explanation
Testcase 1:
9 is the 2nd distinct element present in the array from left to right

Testcase 2:
3 is the first distinct element present in the array from left to right

Testcase 3:
Only 2 distinct elements are present in the array, i.e., 8 and 3
Since there are less than 4 distinct elements, therefore the answer is 0

SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Solution:

```
class Result {
  static int kthDistinctElement(int arr[], int N, int K) {
    // Write your code here
      Map<Integer,Integer> freq=new LinkedHashMap<>();
      for (int i:arr){
          freq.put(i,freq.getOrDefault(i,0)+1);
      }
      ArrayList<Integer> list=new ArrayList<>();
      for (Map.Entry<Integer,Integer> et:freq.entrySet()){
          if(et.getValue()==1){
              list.add(et.getKey());
          }
      }
      if (list.size()<K) return 0;
      return list.get(K-1);
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Maximum Frequency in a sequence

Given a list of integers, find out the number that has the highest frequency. If there are more than one such numbers, output the smaller one.

Input:

First line of input will contain an integer T = number of test cases.
Each test case will contain two lines:
First line will contain an integer N = number of elements in the sequence.
Next line will contain N space separated integers of sequence A.

Output:

For each test case, print on a single line, the smallest number with highest frequency in the sequence.

Constraints

1 <= T <= 100
1 <= N <= 10^5
0 <= A[i] <= 1000

Sample Input

3 // No. of test cases
7
2 4 5 2 3 2 4
6
1 2 1 1 2 1
10
1 1 1 1 1 1 1 1 1 1

Sample Output

2
1
1

Solution:

```
class Result {
  static int maxFrequency(int A[], int n) {
    // Write your code here
     Map<Integer,Integer> frq=new HashMap<>();
      for (int i:A){
          frq.put(i,frq.getOrDefault(i,0)+1);
      }
      int res=0;
      int mx=-1;
      for(Map.Entry<Integer,Integer> et : frq.entrySet()){
          if (et.getValue()>mx || et.getValue() == mx && et.getKey() < res){
              mx=et.getValue();
              res=et.getKey();
          }
      }
      return res;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Check if two arrays are equal or not

Given two array of same size, find out if given arrays are equal or not.
Note: Two arrays are said to be equal if both of them contain same set of elements, although arrangements (or permutation) of elements may be different.
Note : If there are repetitions, then counts of repeated elements must also be same for two array to be equal..
For example, if A = [11, 12, 13] and B = [12, 11, 13] then answer is 1 and,
if A = [11, 12, 13, 12, 13] and B = [12, 11, 13, 12, 13] then answer is 1 and,
if A = [11, 12, 13] and B = [12, 13, 12] then answer is 0.
Complete the function **arraysEqualorNot()** given in the editor, which takes two arrays as input and returns the answer(0/1) as output.
Input Format
The 1st line contains an integer N, the number of elements in A and B.
The 2nd line contains N integers separated by space.
The 3rd line contains N integers separated by space.
Output Format
Print 1 or 0 as per the condition.
Constraints
1 <= N <= 10^5
-50000 <= A[i], B[i] <= 50000
Sample Input
3 // N
11 12 13 // A[]
12 11 13 // B[]
Sample Output
1

Solution:

```
import java.util.HashMap;
import java.util.Map;
class Result {
    static int arraysEqualorNot(int[] A, int[] B) {
        if (A.length != B.length) {
            return 0; // Arrays are not equal if they have different lengths
        }
        Map<Integer, Integer> freqA = new HashMap<>();
        Map<Integer, Integer> freqB = new HashMap<>();
        for (int num : A) {
            freqA.put(num, freqA.getOrDefault(num, 0) + 1);
        }
        for (int num : B) {
            freqB.put(num, freqB.getOrDefault(num, 0) + 1);
        }
        for (Map.Entry<Integer, Integer> entry : freqA.entrySet()) {
            int key = entry.getKey();
            int countA = entry.getValue();
            int countB = freqB.getOrDefault(key, 0);
            if (countA != countB) {
                return 0; // Arrays are not equal if any frequency mismatch
            }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        }
        return 1; // Arrays are equal
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Pair with the given sum

Given a sorted array of **n** elements, and an integer **k**. Find whether there exists any two elements in the array that sums up to the given value k. If such a pair is found, then print "Found", else print "Not Found".
Note: You can not use the *same* index element twice.
Input Format
First line will contain an integer T, denoting the number of test cases.
For each test case:
First line will contain an integer n, denoting the number of elements in the array.
Second line will contain n space separated integers that denote the array elements.
Third line contains an integer k, that denotes the target sum.
Output Format
For each test case, print "Found" if there exists at least one pair whose sum equals to k, else print 'Not Found'.
Constraints
1 <= T <= 10
1 <= n <= 10^5
-(10^9) <= arr[i] <= 10^9
-(10^9) <= k <= 10^9
Sample Input
2 // Test Cases
5 // n (testcase 1)
1 3 4 6 7 // arr[]
9 // k
6 // n (testcase 2)
-3 -1 4 5 7 12 // arr[]
7 // k
Sample Output
Found
Not Found
Explanation:
Testcase 1:
Sum of 3 and 6 in the given array is 9.

Testcase 2:
No pair in the given array has sum equals to 7.

Solution:

```
class Result {
  // Return true if a pair with the sum k is found, else return false
  static boolean pairSum(int arr[], int n, int k) {
    // Write your code her
      Set<Integer> st=new HashSet<>();
      for (int i:arr){
          if (st.contains(k-i)) return true;
          st.add(i);
      }
      return false;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Array Pair Sum Divisibility Problem

Given an array of size n (which is always a even number in this case) and an integer k.
Complete the function that returns true if given array can be divided into pairs such that sum of every pair is divisible by k.
If such pairing of array is possible return 1 else return 0.

Input Format

First Line will contain an integer N denoting the size of array.
Second line contains N integers separated by space as elements of array.
Third line contains an integer k.

Output Format

Print the result as specified above.

Constraints

1 <= N <= 10^5
1 <= a[i] <= 10^5
1 <= k <= 10^5

Sample Input

4
8 4 5 7
4

Sample Output

1

Explanation:

The array can be divided as
(8,4) and (5,7) whose sum is 12, which is divisible by 4.

Solution:

```
class Result
{
  static int isArrayDivisibleInPairs(int arr[], int n, int k){
    // Write your code here
      if (n % 2 != 0) {
            return 0; // Odd number of elements cannot be paired
        }
        Set<Integer> st=new HashSet<>();
        for (int i:arr){
            int r1=i%k;
            int r2=k-r1;
            if (st.contains(r2)){
                st.remove(r2);
            }else if (r1==0 && st.contains(0)){
                st.remove(0);
            }else{
                st.add(r1);
            }
        }
        return (st.size()==0)? 1:0;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Largest subarray with 0 (ZERO) sum

Given an unsorted array of integers. Find the largest sub-array which adds to 0 (ZERO). For example,
Given **A = {11, 2, -2, 10, 1, -4, -7, 19}** , Print "**6**" as the sum 0 is found between indexes 1 and 6. Note that from index 1 to index 2 is also resulting in 0, but we need to print the largest.
If no sub-array found print **-1**.

Input Format

First Line will contain an integer N denoting the number of elements.
Second line contains N integers separated by space.

Output Format

Print the length of the largest sub-array as shown if found otherwise print -1.

Sample Input

8
11 2 -2 10 1 -4 -7 19

Sample Output

6

Solution:

```
class Result
{
  static int largSubArray(int arr[], int n){
    // Write your code here
        HashMap<Integer, Integer> hM = new HashMap<Integer, Integer>();
        int sum = 0, maxLen = -1;
        for (int i = 0; i < n; i++) {
            sum += arr[i];
            if (sum==0){
                maxLen=i+1;
            }
            Integer prev_i = hM.get(sum);
            if (prev_i != null)
                maxLen = Math.max(maxLen, i - prev_i);
            else
                hM.put(sum, i);
        }
        return maxLen;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find K largest elements in array

Given an array of integer elements and an integer k, print the k largest elements from array separated by space in descending sorted order. For example, if array a = [2, 56, 3, 87, 42, 1, 45, 8, 2, 98] and k=3 then output must be 98 87 56.

Input Format:

First line of input contain T = number of test cases. Each test case contain two lines, in first line, total number of elements in array and value of k are provided and next line will contain the elements.

The array of numbers and integer k is given to you.

Note: Do not write the whole program, just complete the functions void printKLargest(int array[], int k) which takes the array and integer k as parameters and print the k largest elements in descending order separated by space.

Note: Do not read any input from stdin/console. Just complete the functions provided. You can write more functions if required, but just above the given function.

Output Format:

Print the k largest element in descending order separated by space.

Do not print the space after last element.

Sample Input

2
10 3
2 56 3 87 42 1 45 8 2 98
6 2
1 87 2 67 3 45

Sample Output

98 87 56
87 67

Solution:

```
//import java.util.*;
static void printKLargest(int array[], int n, int k){
   // Write your code here
   Arrays.sort(array);
   //Arrays.reverse(array);
   for (int i=0;i<k-1;i++){
      System.out.print(array[n-i-1]+" ");
   }
   System.out.print(array[n-k]);
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Convert min Heap to max Heap

Given array representation of a min-heap convert it in a max-heap representation. For example, if array a = [13 15 19 16 18 120 110 112 118] the array must be modified as a = [120 118 110 112 18 19 13 15 16].
The array of numbers is given to you. Do not write the whole program, just complete the functions **void modifyMintoMax(int array[], int n)** which takes the array and total number of elements n as parameters and modify the array elements with **heapify()** to convert it in max-heap.
Note: Do not read any input from stdin/console. Just complete the functions provided. You can write more functions if required, but just above the given function.
Input Format:
First line of input contain T = number of test cases.
Each test case contain two lines, in first line, total number of elements in array and next line will contain the elements.
Output Format:
For each test case, print the max-heap elements separated by space in new lines.
Sample Input
2
9
13 15 19 16 18 120 110 112 118
6
1 2 3 4 5 6
Sample Output
120 118 110 112 18 19 13 15 16
6 5 3 4 2 1

Solution:

```
static void heapify(int array[], int n, int i) {
      while(true){
      int largest = i; // Initialize largest as root
      int left = 2 * i + 1; // Left child
      int right = 2 * i + 2; // Right child
      // If left child is larger than root
      if (left < n && array[left] > array[largest])
         largest = left;
      // If right child is larger than largest so far
      if (right < n && array[right] > array[largest])
         largest = right;
      // If largest is not root
      if (largest != i) {
         int swap = array[i];
         array[i] = array[largest];
         array[largest] = swap;
         i=largest;
      }
      else {
         break;
      }
      }
   }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
static void modifyMintoMax(int array[], int n) {
    // Build max-heap from the given array
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(array, n, i);
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Check if a given tree is max-heap or not

A max-heap is a kind of tree which must satisfy the property as “Every node’s value should be greater than its children’s value”.
Your task is given level wise array representation of a binary tree, check if it is a max-heap or not.
Complete the functions **isMaxHeap()** which takes the array and total number of elements n as parameters and return 1 if array represents a max-heap and 0 otherwise.

Input Format

First line of input contain T = number of test cases. Each test case contain two lines, in first line, total number of elements in tree and next line will contain the elements in level order fashion.

Output Format

For each test case, print 1 if array represents max-heap or 0 otherwise in new lines.

Sample Input

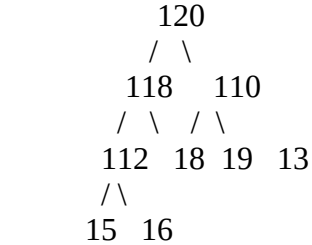
```
2
9
120 118 110 112 18 19 13 15 16
6
1 2 3 4 5 6
```

Sample Output

```
1
0
```

Explanation:

First tree is like below:



It satisfies the properties of max-heap.

Solution:

```
static int isMaxHeap(int array[], int n) {
      // Check if the tree satisfies the max-heap property
      for (int i = 0; i <= (n - 3) / 2; i++) {
         // If left child is greater than the parent
         if (array[2 * i + 1] > array[i]) {
            return 0;
         }
         // If right child exists and is greater than the parent
         if (2 * i + 2 < n && array[2 * i + 2] > array[i]) {
            return 0;
         }
      }
      return 1;
   }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Connect the sticks of different lengths with minimum cost

Given n sticks of different length you have to connect them. The cost of connecting sticks is the sum of their length. You have to find the minimum cost for connecting all sticks. For example, if 3 sticks are there having length respectively 3, 6, 1 then we can connect them in many ways like:
Connect 3 and 6 (Cost 9), then we have 2 sticks of length 9 and 1 connect them (cost 10), So total cost is 19.
Connect 3 and 1 (Cost 4), then we have 2 sticks of length 4 and 6 connect them (cost 10), So total cost is 14.
So your function must return 14 in this case.
Complete the function **connectCost()** which takes the array and total number of sticks as input and returns the minimum cost of connecting all these sticks.

Input Format

First line of input contain T = number of test cases.
Each test case contain two lines, in first line, total number of sticks and next line will contain the lengths of each stick.

Output Format

For each test case, print the total cost of connecting all sticks in new lines.

Constraints

1 <= T <= 10
1 <= n <= 10^5
1 <= lengths[i] <= 1000

Sample Input

2
3
3 6 1
4
4 2 3 6

Sample Output

14
29

Solution:

```
class Result
{
  static int connectCost(int lengths[], int n){
      // Write your code here
      PriorityQueue<Integer> pq=new PriorityQueue<>();
      for (int i:lengths){
          pq.add(i);
      }
      int ans=0;
      while(pq.size()>1){
          int a=pq.poll();
          int b=pq.poll();
          int sum=a+b;
          ans+=sum;
          pq.add(sum);
      }
      return ans;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Implement Priority Queue using Linked List

Implement the priority queue using linked list representation.

Input Format:

First line of input contains an integer Q denoting the number of queries. A query can be of 1 of the below 2 types: -

(a) 1 item p (a query of this type means insert 'item' into the queue with priority p)

(b) 2 (a query of this type means to delete highest priority element from queue and print it, Assume lowest number has highest priority i.e. 0 is highest priority and 9 is lowest).

Next lines of each test case contains Q queries as shown below.

The functions void enqueue() & int dequeue() will take the head of list and modify list and head appropriately. The head pointer is passed by reference to both of them. You are required to complete the methods given.

Output Format:

The output for each test case will be space separated integers having -1 if the queue is empty else the element deleted from the queue.

Sample Input

```
1   // No. of test cases
4   // No. of queries
1   // Query type insert
10  // value
1   // priority
1   // Query type insert
20  // value
0   // priority
2   // Query type delete
2   // Query type delete
```

Sample Output

```
20 10
```

Explanation:

1st query is 1 10 1, which inserts element 10 with priority 1 in queue,
2nd query is 1 20 0, which inserts element 20 with priority 0 in queue,
3rd query is 2, which will delete the highest priority element and print it i.e. 20
4th query is 2, which will delete the highest priority element and print it i.e. 10

Solution:

```
/* class QueueNode
{
  int data;
  int priority;
  QueueNode next;
  public QueueNode(int data, int p)
  {
    this.data = data;
    this.priority = p;
  }
} */
class PQueueLL
{
    public QueueNode front, rear;
    public PQueueLL() {
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        front = null;
        rear = null;
    }
    public void EnQueue(int data, int priority)
    {
        QueueNode temp = new QueueNode(data, priority);
        // If the queue is empty
        if (front == null) {
            front = temp;
            rear = temp;
        } else {
            // Insert at the head if the priority is higher
            if (front.priority > temp.priority) {
                temp.next = front;
                front = temp;
            } else {
                // Traverse the list to find the correct position
                QueueNode current = front;
                while (current.next != null && current.next.priority < temp.priority) {
                    current = current.next;
                }
                // Insert the new node
                temp.next = current.next;
                current.next = temp;
                // If the new node is inserted at the end, update the rear pointer
                if (current == rear) {
                    rear = temp;
                }
            }
        }
    }
    public int DeQueue()
    {
        if (front == null) {
            return -1; // Queue is empty
        }
        QueueNode top = front;
        front = front.next;
        // If the queue becomes empty after DeQueue
        if (front == null) {
            rear = null;
        }
        return top.data;
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Sort an array using heap sort

Given an array of N integers, sort them in ascending order using heap sort. Write the two functions **heapify()** and **heapSort(),** to implement the heap.

Input Format

First line contains the number of elements
Second line contains the elements of array separated by space.

Output Format

Print the elements of sorted array in ascending order.

Constraints

1 <= N <= 10^5
-(10^9) <= arr[i] <= 10^9

Sample Input

7
1 3 5 7 2 4 9

Sample Output

1 2 3 4 5 7 9

Solution:

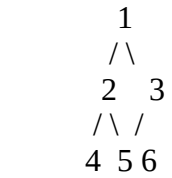
```
import java.util.*;
class Result {
  static void heapify (int array[], int n, int i) {
    // Write your code here
    int large=i;
      int left=2*i+1;
      int right=2*i+2;
      if (left<n && array[large]<array[left]) large=left;
      if (right<n && array[large]<array[right]) large=right;
      if (large!=i){
          int t=array[i];
          array[i]=array[large];
          array[large]=t;
          heapify(array,n,large);
      }
  }
  static void heapSort(int array[], int n) {
    // Write your code here
    for(int i=n/2-1;i>=0;i--)
        heapify(array,n,i);
    for (int i=n-1;i>0;i--){
        int t=array[0];
        array[0]=array[i];
        array[i]=t;
        heapify(array,i,0);
    }
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Create a binary tree from array

Given an array of integer elements, create a binary tree from this array in level order fashion.
Input Format
The array of elements is given and you have to build the tree from it in level order i.e. elements from left in the array will be filled in the tree level wise starting from level 0.
Do not write the whole program, just complete the function buildTree(int arr[], int n) which takes the array and total number of nodes as parameter and return the root of the binary tree.

Note: Do not read any input from stdin/console. Just complete the functions provided. You can write more functions if required, but just above the given function.
þÿOutput Format
Print the tree in inorder.
Constraints
0 <= n <= 10^5
Sample Input
6 // n
1
2
3
4
5
6
Sample Tree



Sample Output
4 2 5 1 6 3

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
// Function to initiate tree building
static Node buildTree(int arr[], int n) {
  if (n<=0 || arr[0]==-1){
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    return null;
  }
  Queue<Node> q=new LinkedList<>();
  Node root=new Node(arr[0]);
  q.add(root);
  int idx=1;
  while(idx<n && !q.isEmpty()){
    Node temp=q.remove();
    if (idx<n && arr[idx]!=-1){
      temp.left= new Node(arr[idx]);
      q.add(temp.left);
    }
    idx++;
    if (idx<n && arr[idx]!=-1){
      temp.right =new Node(arr[idx]);
      q.add(temp.right);
    }
    idx++;
  }
    return root;
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Print binary tree with level order traversal

Given a binary tree, print all nodes of the tree in level order traversal. print nodes at same level separated by space and give new line between every level. **(There should be no space after last node of each level)**

Input

The root node of binary tree is given to you, and you have to complete the function void printLevelWise(Node root) to print the nodes at all levels of the binary tree.

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

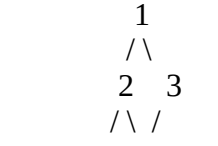
Output

Print the nodes level wise separated by space and new line. There should be no space after last node of each level.

Constraints

0 <= no. of nodes <= 10^5

Sample Input



Sample Output

1
2 3
4 5 6

Solution:

```
static void printLevelWise(Node root) {
    if (root == null) {
        return;
    }
    Queue<Node> q = new LinkedList<>();
    q.add(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            Node node = q.remove();
            System.out.print(node.data);
            if (i<size-1){
              System.out.print(" ");
            }
            if (node.left != null) {
                q.add(node.left);
            }
            if (node.right != null) {
                q.add(node.right);
            }
        }
        System.out.println(); // Move to a new line after finishing each level
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Print nodes at odd levels of the binary tree

Given a binary tree, print all nodes at odd levels of the tree. Assume the root node is at level 1 i.e. odd level.
Complete the function **printOdd()** which will take the root node of the tree as parameter and print the nodes at odd levels of the binary tree.

Input Format

First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

Output Format

Print the nodes at odd levels separated by space.

Constraints

0 <= N <= 10^5

Sample Input

6
1 2 3 4 5 6

Sample Output

1 4 5 6

Explanation:

```
      1          // level-1
     / \
    2   3
   / \ /
  4  5 6         // level-3
```

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  static void printOdd(Node root) {
    if (root == null) {
        return;
    }
    Queue<Node> q = new LinkedList<>();
    q.add(root);
    int level = 1;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            Node node = q.remove();
            if (level % 2 != 0) { // Check if the level is odd
                System.out.print(node.data + " ");
            }
            if (node.left != null) {
                q.add(node.left);
            }
            if (node.right != null) {
                q.add(node.right);
            }
        }
        level++; // Increment the level after processing the current level
    }
}
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Write iterative version of inorder traversal

Given a binary tree, print it in inorder fashion.

Input Format

The root node of binary tree is given to you, and you have to complete the function void printInorder(Node root) to print the nodes of tree. (The function should use iteration not recursion).

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

Output Format

Print the nodes of tree separated by space.

Constraints

0 <= N <= 10^5

Sample Input

```
      1
     / \
    2   3
   / \ /
  4  5 6
```

Sample Output

4 2 5 1 6 3

Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
static void printInorder(Node root)
{
     if (root==null) return;
    printInorder(root.leftChild);
    System.out.print(root.data+" ");
    printInorder(root.rightChild);
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Complete the inorder(), preorder() and postorder() functions for traversal with recursion

Given a binary tree, print it in inorder, preorder and postorder fashion with recursion.

Input

The root node of binary tree is given to you, and you have to complete the function void inorder(Node root), void preorder(Node root) & void postorder(Node root) to print the nodes of tree. (The function should use recursion).

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

Output

Print the nodes of tree separated by space by all three traversals in new lines.

Sample Input

6
1 2 3 4 5 6

Above input corresponds to below tree.

```
      1
     / \
    2   3
   / \ /
  4  5 6
```

Sample Output

4 2 5 1 6 3
1 2 4 5 3 6
4 5 2 6 3 1

Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
static void inorder(Node root)
{
  if (root==null){
    return ;
  }
  inorder(root.leftChild);
  System.out.print(root.data+" ");
  inorder(root.rightChild);
}
static void preorder(Node root)
{
    if (root==null){
    return ;
  }
  System.out.print(root.data+" ");
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
  preorder(root.leftChild);
  preorder(root.rightChild);
}
static void postorder(Node root)
{
    if (root==null){
    return ;
  }
  postorder(root.leftChild);
  postorder(root.rightChild);
  System.out.print(root.data+" ");
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Construct tree from given inorder and post order traversal

Given two array representing the inorder and postorder traversal of a binary tree, construct the tree and return its root node.

Input

Two arrays for traversals and total number of nodes are given and you needs to write a function Node buildTree(int in[], int post[], int N), which accepts the inorder and postorder traversals of a binary tree and the number of nodes in the tree. The function returns the root node of the constructed tree.

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

Output

The nodes of tree will be printed separated by space using preorder traversal in new lines.

Sample Input

```
6 // Number of nodes
4 // inorder traversal
2
5
1
6
þÿ3
4 // Postorder traversal
5
2
6
3
1
```

Sample Output

```
1 2 4 5 3 6
```

Solution:

```
class Result {
    static int postidx;
    static int findpos(int[] in,int st,int n,int ele){
        for (int i=st;i<n;i++){
            if (in[i]==ele){
                return i;
            }
        }
        return -1;
    }
    static Node buildtreehelper(int[] in ,int[] pos,int st,int end){
        if (postidx<0 || st>end){
            return null;
        }
        int ele=pos[postidx--];
        int posi=findpos(in,st,in.length,ele);
        Node root=new Node(ele);
        root.rightChild=buildtreehelper(in,pos,posi+1,end);
        root.leftChild=buildtreehelper(in,pos,st,posi-1);
        return root;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    }
  static Node buildTree(int in[], int post[], int N) {
    // Write your code here
      postidx=N-1;
      Node root=buildtreehelper(in,post,0,N-1);
      return root;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Count the number of leaf and non-leaf nodes in a binary tree

A leaf in a tree is a node which has no children, i.e. for a binary tree, both its left and right point to NULL. Given a binary tree count the number of leaf and non-leaf nodes in it. Complete the functions **countLeafs()** & **countNonLeafs()** which takes the address of the root node as parameter and return the count of the leaves and non-leafs respectively.

þÿInput Format

First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.

Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

Output Format

For each test case, print the number of leaf and non-leaf nodes separated by space in new lines.

Constraints

0 <= N <= 10^5

Sample Input

7

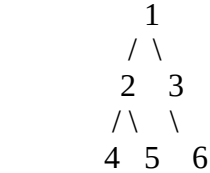
1 2 3 4 5 -1 6

Sample Output

3 3

Explanation:

The above tree is:



Solution:

```
class Result {
  static int helperCntleaf(Node root){
    if (root==null){
      return 0;
    }
     int lft= helperCntleaf(root.left);
    if (root.left==null && root.right==null){
      return 1;
    }
    int rght= helperCntleaf(root.right);
    return lft+rght;
  }
static int helperNonCntleaf(Node root) {
    if (root == null) {
        return 0;
    }
    if ((root.left != null || root.right != null)) {
        return 1 + helperNonCntleaf(root.left) + helperNonCntleaf(root.right);
    }
    return 0;
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
  static int countLeafs(Node root) {
    int res= helperCntleaf(root);
    return res;
  }
  static int countNonLeafs(Node root) {
    int res= helperNonCntleaf(root);
    return res;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Print all paths to leaves and their details of a binary tree

Given a binary tree print all paths from root node to leaf nodes with their respective lengths and total number of paths in it.
The root node of binary tree is given to you. A path from root to leaf in a tree is a sequence of adjacent nodes from root to any leaf node.
Do not write the whole program, just complete the function **void printAllPaths(Node root)** which takes the address of the root as a parameter and print all details as shown below.
Note: If the tree is empty, do not print anything.

Input Format

First line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

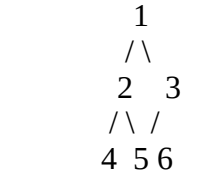
Output

For each test case, print the paths with their length and total paths in new lines.

Constraints

0 <= N <= 10^5
0 <= node data <= 1000

Sample Input



Sample Output

1 2 4 2
1 2 5 2
1 3 6 2
3

Explanation:

First path is from 1 to 4 having 2 edges, so 1 2 4 is path and 2 is length of it.
Similarly for other two leaf nodes. And at last line total number of paths are printed.

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
 public static int countleaf(Node root){
```

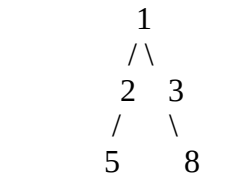

SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
  if (root==null){
    return 0;
  }
  int lft = countleaf(root.left);
  if (root.left ==null && root.right == null){
    return 1;
  }
  int riht = countleaf(root.right);
  return lft+riht;
}
public static void printAllPaths(Node root) {
      if (root== null){
        return ;
      }
    List<Integer> path = new ArrayList<>();
    printAllPathsHelper(root, path);
      int res=countleaf(root);
      System.out.println(res);
}
// Helper function to print all paths
public static void printAllPathsHelper(Node root, List<Integer> path) {
    if (root == null) {
        return;
    }
    path.add(root.data);
    if (root.left == null && root.right == null) {
        // This is a leaf node
        printPath(path);
    } else {
        // Recur for left and right subtrees
        printAllPathsHelper(root.left, path);
        printAllPathsHelper(root.right, path);
    }
    // Remove the current node from the path
    // This will allow backtracking
    path.remove(path.size() - 1);
}
// Function to print the path and its length
public static void printPath(List<Integer> path) {
    for (int i = 0; i < path.size(); i++) {
        System.out.print(path.get(i));
        if (i != path.size() - 1) {
            System.out.print(" ");
        }
    }
    System.out.print(" " + (path.size() - 1));
    System.out.println();
}
```

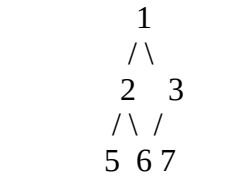

SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find the right node of a given node

Given a binary tree and a key in it, find the node which is on same level and on right of this given key. If no such node present return -1.
Complete the function **findRightSibling()** which takes the address of the root node and an integer key as a parameter and return the right sibling (if exists, otherwise return -1).
þÿInput Format
First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.
Third line contains an integer key whose right sibling is desired.
Output Format
Print the right sibling if any, otherwise print -1.
Sample Input 1
7
1 2 3 5 -1 -1 8
5
Sample Output 1
8
Explanation 1



þÿFor 5, the right sibling is 8.
Sample Input 2
6
1 2 3 5 6 7
7
Sample Output 2
-1
Explanation 2



For 7, there is no node present in its right on the same level.
Therefore, the answer is -1.

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  static ArrayList<Integer> buildarr(Node root){
    ArrayList<Integer> al= new ArrayList<>();
    if (root==null){
      return al;
    }
    Queue<Node> q= new LinkedList<>();
    q.add(root);
      q.add(null);
    while(!q.isEmpty()){
      Node temp=q.remove();
        if (temp==null){
            if (!q.isEmpty()){
                al.add(-1);
                q.add(null);
            }
        }else{
            al.add(temp.data);
      if (temp.left!=null){
        q.add(temp.left);
      }if (temp.right!=null){
        q.add(temp.right);
      }
        }
    }
    return al;
  }
  static int findRightSibling(Node root, int key) {
    // Write your code here
    ArrayList<Integer> arr=buildarr(root);
      //System.out.println(arr);
    for (int i=0;i<arr.size()-1;i++){
      if (arr.get(i)==key){
        return arr.get(i+1);
      }
    }
    return -1;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Convert a binary tree into its mirror tree

Given a binary tree, convert it in its mirror image. Mirror of a Binary Tree is another Binary Tree in which left and right children of all non-leaf nodes are interchanged.

Input

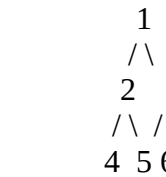
The root node of binary tree is given to you. Do not write the whole program, just complete the function findMirror(Node root) which takes the address of the root as a parameter and change the tree in its mirror image.

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

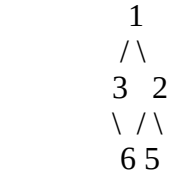
Output

For each test case, print the tree in inorder in new lines.

Sample Input



Sample Output



Inorder: 3 6 1 5 2 4

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
static Node findMirror(Node root) {
   if (root == null) {
      return null;
   }
   Node temp = root.left;
   root.left = root.right;
   root.right = temp;
   findMirror(root.left);
```

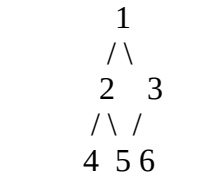

SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    findMirror(root.right);
    return root;
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Print cousins of a given node in Binary Tree

Given a binary tree and a key value from this tree, print all the cousins of this node separated by space. If no cousin exists print -1.
Complete the function **printCousins()** which takes the address of the root node and a key k as a parameter and print the cousins of k separated by space or -1 if no cousin exists.
Input Format
First line contains the total number of nodes, second line contains the node labels separated by space. Third line contains an integer key k whose cousins are desired.
Output Format
For each test case, print the cousins of given key separated by space in new lines.
Sample Input
6
1 2 3 4 5 6
2
Sample Output
-1
Explanation:



The parent of node 2 is 1, who have no siblings, no cousins for node 2.
If key is 6, then 2 is the sibling of parent i.e. 3. So the cousins are 4 and 5þÿ.

Solution:

```
import java.util.*;
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
class Result {
  public static void printCousins(Node root, int k) {
    if (k==0 ||root == null) {
          System.out.println("-1");
      return;
    }
    Queue<Node> q = new LinkedList<>();
    q.add(root);
    boolean found = false;
    while (!q.isEmpty() && !found) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            Node node = q.poll();
            // Check if the parent of the node has the target node as a child
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
            if ((node.leftChild != null && node.leftChild.data == k) || (node.rightChild != null
&& node.rightChild.data == k)) {
                found = true;
                continue;
            }
            if (node.leftChild != null) {
                q.add(node.leftChild);
            }
            if (node.rightChild != null) {
                q.add(node.rightChild);
            }
        }
        if (found) {
            // Print cousins
              int flag=0;
            while (!q.isEmpty()) {
                Node cousin = q.poll();
                  flag=1;
                System.out.print(cousin.data + " ");
            }
            if (flag==0){
              System.out.println("-1");
            }
            System.out.println();
            return;
        }
    }
    // If no cousins found, print -1
    System.out.print("-1");
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Print nodes in a top view of Binary Tree

Given a binary tree, print top view of the binary tree.
For Example:
[image:https://lh4.googleusercontent.com/6ciAz3U8GM6G1FoGxS5R-6xadLRYegCubYTrdNglg5hV55TT4fzTEes9Mkc6MZMV5M0_93uhVw41jc6QqQo8M6ast5iF7Ms_fJIggUTS447inFfYobB20pjcAE_Inw]

Top view of the given binary tree will be: **4 2 1 3 9**
Complete the function **printTopView()** which takes the address of the root as a parameter and print the tree from top view.
Input Format
First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.
Output Format
For each test case, print the tree nodes from top view separated by space in new lines.
Sample Input
7
1 2 3 4 5 -1 6
Sample Output
4 2 1 3 6
Explanation:

```
      1
     / \
    2   3
   / \   \
  4   5   6
```

So, from top, first node is the left most node i.e. 4, and then 2, 1, 3 and lastly 6.
þÿNote that, 5 is not seen from top.

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Pair{
   Node first;
   int sec;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    public Pair(Node f,int s){
        this.first=f;
        this.sec=s;
    }
}
class Result {
  static void printTopView(Node root) {
    // Write your code here
      if (root==null) return ;
      ArrayList<Integer> arr=new ArrayList<>();
      Queue<Pair> q=new LinkedList<>();
      Map<Integer,Integer> mp=new TreeMap<>();
      q.add(new Pair(root,0));
      while(!q.isEmpty()){
          Pair temp=q.poll();
          Node fst=temp.first;
          int hd=temp.sec;
          if (!mp.containsKey(hd)){
              mp.put(hd,fst.data);
          }
          if (fst.left!=null){
              q.add(new Pair(fst.left,hd-1));
          }
          if (fst.right!=null){
              q.add(new Pair(fst.right,hd+1));
          }
      }
      arr.addAll(mp.values());
      for (int i:arr){
          System.out.print(i+" ");
      }
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find out if the tree can be folded or not

Given a binary tree, find out if it can be folded or not. A tree can be folded if left and right sub-trees of root are structure wise mirror images.
Note: A tree with zero or one node is foldable inherently.
Complete the function **isFoldable()** which takes the address of the root as parameter and return 1 if tree can be folded and 0 otherwise.

Input Format

First line contains an integer T, denoting the number of test cases.
For each test case:
First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

Output Format

For each test case, print 1 if tree is foldable and 0 otherwise, in new lines.

Sample Input 1

```
    1
   / \
  2   3
```

Sample Output 1

1

Sample Input 2

```
     1
    / \
   2   3
  / \
 4  5
```

Sample Output 2

0

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
    static int areSameTree(Node r1, Node r2) {
  // Write your code here
       if (r1 == null && r2 == null) {
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
            return 1;
        }
        // One tree is empty, and the other is not
        if (r1 == null || r2 == null) {
            return 0;
        }
        // Recursively check for left and right subtrees
        int left = areSameTree(r1.left, r2.right);
        int right = areSameTree(r1.right, r2.left);
        // Return true only if current nodes match and both subtrees are identical
        return (left==1 && right==1) ? 1:0;
    }
    static int isFoldable(Node root) {
    // Write your code here
       if (root == null) {
            return 1;  // An empty tree is foldable
        }
       return areSameTree(root.left, root.right);
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Given two trees are identical or not

Given two binary trees, find out both are same or not. Two trees are considered same if they have same nodes and same structure.
So complete the function **areSameTree()** which takes the address of the root nodes of two trees as parameters and return **1** if both are same and **0** otherwise.

Input Format

The first line contains the number of testcases, T.
For each Testcase:
First line contains the number of nodes in first tree and second line contains the node labels in level-wise order.
Third line contains the number of nodes in second tree and fourth line contains the node labels in level-wise order.

Output Format

For each test case, print 1 if both tree are same and 0 otherwise.

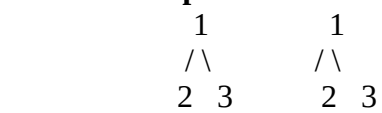
Sample Input 1

1 // Number of testcases
3
1 2 3
3
1 2 3

Sample Output 1

1

Explanation 1



As both the trees have same number of nodes and all same labels so trees are same.

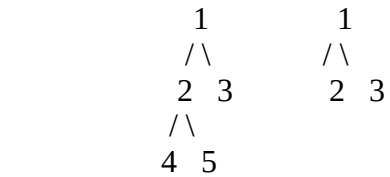
Sample Input 2

1 // Number of testcases
5
1 2 3 4 5
3
1 2 3

Sample Output 2

0

Explanation 2



In this case, trees are not same

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  /*
   * @Input
   *   t1, t2 -> Given the root node of two binary trees
   *
   * @Output
   *   Return 1 if the two binary trees are identical, else return 0
   */
  static int areSameTree(Node r1, Node r2) {
  // Write your code here
      if (r1 == null && r2 == null) {
          return 1;
       }
       // One tree is empty, and the other is not
       if (r1 == null || r2 == null) {
          return 0;
       }
       // Check if the current nodes have the same data
       int val = (r1.data == r2.data)?1:0;
       // Recursively check for left and right subtrees
       int left = areSameTree(r1.left, r2.left);
       int right = areSameTree(r1.right, r2.right);
       // Return true only if current nodes match and both subtrees are identical
       return (left==1 && right==1 && val==1) ? 1:0;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find maximum depth or height of a binary tree

Height of a tree is defined as the length of the longest downward path from root node to any leaf. If tree is empty, height is considered as -1 and for tree with only one node height is considered as 0.
Your task is that given a binary tree, find out the maximum depth of tree (also called height of tree).
Complete the function **treeHeight()** which takes the address of the root node of tree as parameter and return the height of the tree.

Input Format

First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

Output Format

For each test case, print the height of tree.

Constraints

0 <= N <= 10^5

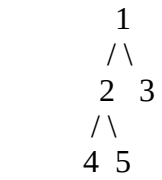
Sample Input

5
1 2 3 4 5

Sample Output

2

Explanation:



The longest path are 1-2-4 and 1-2-5, both have a length of 2, so tree height is 2.

Solution:

```
import java.util.*;
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
  static int treeHeight(Node root) {
  // Write your code here
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    if (root==null){
      return -1;
    }
    int lft=treeHeight(root.left);
    int rght=treeHeight(root.right);
    int res=Math.max(lft,rght)+1;
    return res;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Evaluation of expression tree

Given a expression binary tree with 4 binary operators (+, -, *, /) and integer operands, evaluate it and print the answer.
Complete the function **evaluateTree()** which takes the address of the root node of tree as parameter and return the result of expression.
Note: The nodes with operators are given as ASCII codes of these operators (e.g. 42 for *(multiply), 43 for +(addition), 45 for -(subtraction) & 47 for /(division)).
Input Format
First line contains the total number of nodes and second line contains the node labels separated by space.
Output Format
For each test case, print the result of expression, in new lines.
Sample Input
7
43 42 43 4 5 2 4
Sample Output
26
Explanation:

```
     +
    / \
   *   +
  / \  / \
 4  5 2  4
```

Nodes which are having child nodes are the operator nodes and leaf nodes are the operand nodes to distinguish between.

Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
class Result {
  static int evaluateTree(Node root) {
    if (root == null) {
        return 0;
    }
    // If the current node is a leaf node, return its value
    if (root.leftChild == null && root.rightChild == null) {
        return root.data;
    }
    // Recursively evaluate left and right subtrees
    int leftEval = evaluateTree(root.leftChild);
    int rightEval = evaluateTree(root.rightChild);
    // Apply the operator at the current node
    switch (root.data) {
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
            case 42: // '*'
                return leftEval * rightEval;
            case 43: // '+'
                return leftEval + rightEval;
            case 45: // '-'
                return leftEval - rightEval;
            case 47: // '/'
                return leftEval / rightEval;
            default:
                return -1;
        }
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Given binary tree is binary search tree or not

A binary tree is called binary search tree if it holds following three properties: -
a. The left subtree of a node contains nodes whose keys are less than that node's key.
b. The right subtree of a node contains nodes whose keys are greater than that node's key.
c. Both the left and right subtrees must also be binary search trees.
Your task is that, given a binary tree check if it is binary search tree or not. Complete the function **isBinarySearchTree()** which takes the address of the root node of tree as parameter and return **1** if the tree is binary search tree and **0** otherwise.

Input Format

First line contains an integer T, denoting the number of test cases.
For each test case:
First Line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.

Output Format

For each test case, print 0 or 1, in new lines.

Constraints

1 <= T <= 10
0 <= N <= 10^5þÿ

Sample Input

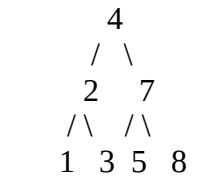
1 // testcases
7 // N
4 2 7 1 3 5 8

Sample Output

1

Explanation:

Given tree is:



Which satisfies the property of binary search tree, so return value is 1.

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
 */
class Result {
  static boolean isbst(Node root,int min ,int max){
    if(root==null){
      return true;
    }
    if (root.data>min && root.data<max){
      boolean left = isbst(root.left,min,root.data);
      boolean right =isbst(root.right,root.data,max);
      return left && right;
    }
    else{
      return false;
    }
  }
  static int isBinarySearchTree(Node root) {
    // Write your code here
    return (isbst(root,Integer.MIN_VALUE,Integer.MAX_VALUE)) ? 1:0;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find the kth smallest element in the binary search tree

Given a binary search tree and a number **k**, print the kth smallest number in tree.

Input

The root node of binary search tree is given to you. Do not write the whole program, just complete the function int kSmallest(Node root, int k) which takes the address of the root node of tree and an integer k as parameters and return the kth smallest number from tree.

Note: Do not read any input from stdin/console. Just complete the function provided. You can write more functions if required, but just above the given function.

Output

Print the kth smallest number from the binary search tree.

Sample Input

```
      4
     / \
    2   7
   / \  / \
  1  3 5  8
```

k = 4
k = 6

Sample Output

4
7

Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
  static void helper(Node root,List<Integer> arr){
    if (root==null){
      return ;
    }
    helper(root.left,arr);
    arr.add(root.data);
    helper(root.right,arr);
  }
  static int kSmallest(Node root, int k) {
    // Write your code here
    List<Integer> arr= new ArrayList<>();
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
  helper(root,arr);
  int ans=0;
  for(int i=0;i<k;i++){
    ans=arr.get(i);
  }
  return ans;
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Convert Level Order Traversal to BST

Given an array of integer elements representing the level order traversal of a binary search tree, create the binary search tree from this array.
Write the function **buildSearchTree()** which takes the array and total number of nodes as parameters and return the root of the binary search tree.

Input Format

First line contains the number of elements in the array(containing the level order traversal) and second line contains the elements separated by space.

þÿOutput Format

Print the tree with inorder traversal.

Sample Input

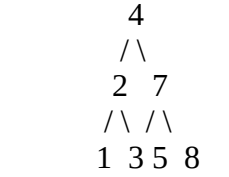
7
4 2 7 1 3 5 8

Sample Output

1 2 3 4 5 7 8

Explanation:

The tree from above level order traversal will be:



Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
class Result {
  static Node insert(Node root,int d){
    if (root==null){
      return new Node(d);
    }
    if (root.data > d){
      root.leftChild=insert(root.leftChild,d);
    }if (root.data<= d) {
      root.rightChild=insert(root.rightChild,d);
    }
    return root;
  }
  static Node buildSearchTree(int t[], int n) {
    Node root = null;
    // Complete the function body.
    if (n==0){
      return root;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    }
    root=new Node(t[0]);
    for (int i=1;i<n;i++){
      root=insert(root,t[i]);
    }
    return(root);
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find a lowest common ancestor of a given two nodes in a binary search tree

The Lowest Common Ancestor (LCA) of two nodes in a Binary Search Tree (BST) is defined as the deepest node from the root which lies in the path of both the nodes from the root.
So, given a binary search tree, find the Lowest Common Ancestor of two given nodes in it.
Complete the function **lowestCommonAncestor()** which takes the address of the root node of tree and two integer keys as parameters and return the lowest common ancestor of these two nodes (For empty tree return -1).
You can assume that both the given keys are present in the tree and are different from each other.

Input Format

First line contains an integer N, denoting the number of integers to follow in the serialized representation of the tree.
Second line contains N space separated integers, denoting the level order description of left and right child of nodes, where -1 signifies a NULL child.
Third line contains two integers k1 and k2 separated by space.

Output Format

Print the lowest common ancestor of given two nodes from the binary search tree.

Constraints

0 <= N <= 10^5

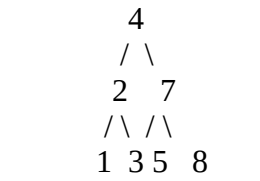
Sample Input

```
7
4 2 7 1 3 5 8
1 5
```

Sample Output

```
4
```

Explanation:



Solution:

```
/*
 *  class Node {
 *    int data;
 *    Node left;
 *    Node right;
 *    public Node() {
 *      data = 0;
 *    }
 *    public Node(int d)  {
 *      data = d;
 *    }
 *  }
 *
 *  The above class defines a tree node.
 */
class Result {
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
  static int lowestCommonAncestor(Node root, int k1, int k2) {
  // Write your code here
    if (root==null){
      return -1;
    }
    if (k1==root.data || k2==root.data){
      return root.data;
    }
    if (k1<root.data && k2<root.data){
      return lowestCommonAncestor(root.left,k1,k2);
    }
    if (k1>root.data && k2>root.data){
      return lowestCommonAncestor(root.right,k1,k2);
    }
    return root.data;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find the floor and ceil of a key in binary search tree

Given a binary search tree, find the floor and ceil of an key in the tree. They are defined as below:
floor value of k: largest node value which is smaller than or equal to k.
ceil value of k: smallest node value which is greater than or equal to k.
Complete the functions **floorOf()** & **ceilOf()** which takes the address of the root node of tree and an integer key as parameters and return the floor and ceil of that key respectively, and -1 if not found.

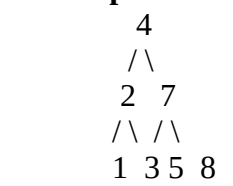
Input Format:
First line contains the number of nodes, second line contains the node labels in level-wise order. Third line contains an integer k whose floor and ceil are desired.

Output Format:
Print the floor and ceil node values of given key from the binary search tree. If not exists, print -1.

Sample Input
7
4 2 7 1 3 5 8
2

Sample Output
2 2

Explanation:



For 2, As 2 is itself present in the tree so 2 will be the floor and ceil of itself.
Furthermore, for 6, the largest node value which is smaller than or equal to 6 is 5, and smallest node value which is greater than or equal to 6 is 7.
And for 9, the largest node value which is smaller than or equal to 9 is 8, and smallest node value which is greater than or equal to 9 is not in the tree so -1 will be printed.

Solution:

```
/* class Node {
  int data; // data used as key value
  Node leftChild;
  Node rightChild;
  public Node()  {
    data=0;  }
  public Node(int d)  {
    data=d;  }
 } Above class is declared for use. */
class Result {
  static int floorOf(Node root, int key) {
    int floor =-1;
    while(root!=null){
      if (root.data==key){
        return key;
      }
      if (root.data>key){
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        root=root.leftChild;
      }else{
        floor=root.data;
        root=root.rightChild;
      }
    }
    return floor;
  }
  static int ceilOf(Node root, int key) {
    int ceil=-1;
    while(root!=null){
      if (root.data==key){
        return key;
      }
      if (root.data>key){
        ceil=root.data;
        root=root.leftChild;
      }else{
        root=root.rightChild;
      }
    }
    return ceil;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Fractional Knapsack problem

The knapsack problem or rucksack problem is a problem in combinatorial optimization: Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.
In this problem we will solve a variant of it, in which the items may have a fractional decision i.e. you can pick the full item or partial depending on the capacity you have. Partial items are allowed to fill the bag at fullest. This is also known as fractional knapsack problem.
Given two integer arrays values[0..n-1] and weight[0..n-1] which represent values and weights associated with n items respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of values[] such that sum of the weights of this subset is equal to W.

Input Format:

First Line will contain an integer N denoting the number of items.
Second line contains N integers separated by space as values of N items.
Third line contains N integers separated by space as weights of N items.
Fourth line contains an integer denoting the capcacity of knapsack.

Output Format:

Print the maximum value that can be earned with above knapsack.

Constraints:

1 <= N <= 10^5
1 <= val[i] <= 10^3
1 <= weight[i] <= 10^3
1 <= capacity <= 10^7

Sample Input

3
25 24 15
18 15 10
20

Sample Output

31.50

Explanation:

Pick 2nd item in full and 3rd item half which makes total weight as (15 + 10/2) = 20 and profit as (24 + 15/2) = 31.5

Solution:

```
class Pair{
   double val,wt,ratio;
   public Pair(int f,int s){
      this.val=f;
      this.wt=s;
      this.ratio=((double)val/(double)wt);
   }
   public String toString(){
      return "Items:" + "["+val+" "+wt+" "+ ratio+"]";
   }
}
class Result
{
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
 static double fractionalKnapsack(int val[], int weight[], int n, int capacity)
 {
    Pair[] items=new Pair[n];
    for(int i=0;i<n;i++){
       items[i]=new Pair(val[i],weight[i]);
    }
    Arrays.sort(items,new Comparator<Pair>(){
       public int compare(Pair i,Pair j){
          return Double.compare(j.ratio,i.ratio);
       }
     });
    double totalval=0.0;
    for (int i=0;i<n;i++){
       if (capacity - items[i].wt >=0){
          capacity -= items[i].wt;
          totalval += items[i].val;
       }else{
          double frac = (double)capacity/items[i].wt;
          totalval+=(frac*items[i].val);
          break;
       }
    }
    return totalval;
 }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Interval scheduling Problem

Interval is defined as a span of time with start and end time. You are given **N** intervals with their start and finish times.
Two intervals can overlap each other if they have any time in common, e.g. Interval(5, 15) and Interval(10,20) are overlapping as time 10 to 15 is shared between them,
whereas Interval(5, 15) and Interval(20, 30) are non-overlapping as they do not have any common time.
Your task is to find the maximum number of intervals that can be scheduled, obviously they need to be non-overlapping.

Input Format:
The 1st line contains an integer N, the number of intervals.
The 2nd line contains N integers separated by space denoting starting times of N intervals.
The 3rd line contains N integers separated by space denoting finishing times of N intervals.

Output Format:
Print the number of non-overlapping intervals that can be scheduled.

Constraints:
1 <= N <= 10^5

Sample Input
4
10 25 12 40
26 32 22 50

Sample Output
3

Explanation:
We can start with the interval no 3 i.e. having start time 12 and finish time 22, then
with the interval no 2 i.e. having start time 25 and finish time 32 and last with the interval no 4 i.e. having start time 40 and finish time 50.

Solution:

```
class sch{
    int st;
    int et;
    public sch(int st,int et){
        this.st=st;
        this.et=et;
    }
    public String toString(){
        return "Jobs "+ st +"-> " + et;
    }
}
class Interval implements Comparable<Interval> {
    int start, end;
    public Interval(int start, int end) {
        this.start = start;
        this.end = end;
    }
    @Override
    public int compareTo(Interval other) {
        return this.end - other.end; // Sort by finish time
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
    }
}
class Result {
  static int intervalScheduling(int[] start, int[] end) {
      int n = start.length;
        List<Interval> intervals = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            intervals.add(new Interval(start[i], end[i]));
        }
        // Sort intervals by finish time
        Collections.sort(intervals);
        int count = 0;
        int lastEndTime = Integer.MIN_VALUE;
        for (Interval interval : intervals) {
            if (interval.start >= lastEndTime) {
                lastEndTime = interval.end;
                count++;
            }
        }
        return count;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Job Scheduling with deadlines

Mr. Amit have some jobs to be scheduled. Each job must meet the deadline to be counted as completed i.e. scheduling a job after its deadline is of no use.
Each job has some profit associated with it. Mr. Amit wants to schedule the as much jobs as he can and also want to earn the maximum profit.
Note: Each job needs 1 time unit for execution.

Input Format:

The 1st line contains an integer N, the number of jobs.
The 2nd line contains N integers separated by space denoting deadline times of N jobs.
The 3rd line contains N integers separated by space denoting profits of N jobs.

Output Format:

Print the maximum profit that can be earned.

Constraints:

1 <= N <= 10^5
1 <= Deadline <= 100
1 <= Profit <= 500

Sample Input

4
2 1 1 2
6 8 5 10

Sample Output

18

Explanation:

Job2 can be scheduled to execute at time 1 with profit 8.
Job4 can be scheduled to execþÿute at time 2 with profit 10.
So maximum profit that can be earned is 18.

Solution:

```
class Result {
  static int jobScheduling(int[] deadlines, int[] profits) {
    // Write your code here
      int n =deadlines.length;
      int maxi = Integer.MIN_VALUE;
      for(int i=0;i<n;i++)
      {
         maxi = Math.max(maxi,deadlines[i]);
      }
      boolean[] slots = new boolean[maxi+1];
      int[][] arr = new int[n][2];
      for(int i=0;i<n;i++)
      {
         arr[i][0] = deadlines[i];
         arr[i][1] = profits[i];
      }
      Arrays.sort(arr,(a,b) -> b[1]-a[1]);
      int total = 0;
      for(int i=0;i<n;i++)
      {
         for(int j=arr[i][0];j>0;j--)
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
            {
                if(slots[j]==false)
                {
                    total+=arr[i][1];
                    slots[j] = true;
                    break;
                }
            }
        }
        return total;
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Activity Selection Problem

Activity is defined as a interval of time for its execution. Every activity has a start time and a finish time. You are given **N** activities with their start and finish times.
Select the maximum number of activities that can be performed by a single person, assuming that a person can only work on a single activity at a time.
Note: Remember that you should take care of overlapping of activity times, and always choose the very next activity instead of jump if that does not make any difference to the maximum number of activities.

Input Format:
The 1st line contains an integer N, the number of activities.
The 2nd line contains N integers separated by space denoting starting times of N activities.
The 3rd line contains N integers separated by space denoting finishing times of N activities.

Output Format:
Print the starting time of picked activities in sorted order separated by space.

Constraints:
1 <= N <= 10^5

Sample Input
3 // N
20 22 30 // start[]
30 35 40 // finish[]

Sample Output
20 30

Explanation:
First activity will finish at time 30, thereafter we can not select the 2nd activity.
However we can pick the 3rd activity as it will start after 1st will over.
So in this way we can select a maximum of 2 activities and their starting time are printed.

Solution:

```
class Activity{
    int fst;
    int sec;
    public Activity(int f,int s){
        this.fst=f;
        this.sec=s;
    }
}
class Result {
  // Print the starting time of selected activities in sorted order separated by space
  static void activitySelection(int[] start, int[] finish) {
      // Write your code here
      int n=start.length;
      Activity[] arr= new Activity[n];
      for(int i=0;i<n;i++){
          arr[i]=new Activity(start[i],finish[i]);
      }
      Arrays.sort(arr,(i,j)-> i.sec - j.sec);
      int i=0;
      System.out.print(arr[i].fst+" ");
      for (int j=1;j<n;j++){
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
            if (arr[j].fst >= arr[i].sec){
                System.out.print(arr[j].fst+" ");
                i=j;
            }
        }
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Count number of ways to cover a distance

Mr. Amit has a puzzle to solve. He needs your help. The puzzle is to find total number of ways to cover a distance of D steps where he can take on only 1,2,....,K steps in one go. For example, if D=3 & K=3, he can do it in 4 ways: {1+1+1, 1+2, 2+1, 3}, total 4 ways to complete 3 steps.

Input Format

First Line will contain an integer D denoting the distance to be covered.
Second Line will contain an integer K denoting the steps can be taken at most in one go.

Output Format

Print the total number of ways.

Sample Input

3
3

Sample Output

4

Solution:

```
import java.util.*;
class Result
{
  static int helper(int d, int k,int[] dp){
    // Write your code here
      if (d==0){
          return 1;
      }
      if (dp[d]!=-1){
          return dp[d];
      }
      int cnt=0;
      for (int i=1;i<=k;i++){
          if (d-i>=0)
           cnt+=helper(d-i,k,dp);
      }
      dp[d]=cnt;
      return dp[d];
  }
    static int totalWaysToDistance(int d, int k){
    // Write your code here
        if (k==0) return 0;
          int[] dp=new int[d+1];
        Arrays.fill(dp,-1);
        return helper(d,k,dp);
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Minimum Cost Path to last element of matrix

Given a cost matrix cost[M][N], write a function that returns cost of minimum cost path to reach (M, N) from (0, 0). Each cell of the matrix represents a cost to traverse through that cell. Total cost of a path to reach (M, N) is sum of all the costs on that path (including both source and destination). You can only traverse down, right and diagonally lower cells from a given cell, i.e., from a given cell (i, j), cells (i+1, j), (i, j+1) and (i+1, j+1) can be traversed. You may assume that all costs are positive integers.

Input Format

First Line will contain two integers M and N denoting the size of matrix.
Second line contains M*N integers as matrix elements row-wise separated by space.

Output Format

Print the minimum cost to reach from source to destination in matrix.

Sample Input

3 3
1 2 3 4 8 2 1 5 3

Sample Output

8

Explanation

We can traverse from elements (0,0) -> (0,1) -> (1,2) -> (2,2)

Solution:

```
class Result
{
  static int minCostPath(int cost[][], int m, int n){
    // Write your code here
      for(int i=1;i<n;i++){
         cost[0][i]+=cost[0][i-1];
      }
      for (int i=1;i<m;i++){
         cost[i][0]+=cost[i-1][0];
      }
      for (int i=1;i<m;i++){
         for (int j=1;j<n;j++){
            cost[i][j]+=Math.min(cost[i-1][j],Math.min(cost[i][j-1],cost[i-1][j-1]));
         }
      }
      return cost[m-1][n-1];
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Longest Common Subsequence (LCS)

Overlapping Subproblems and Optimal Substructure properties are the prerequisites for solving a problem with dynamic programming. In this problem you need to figure them out. LCS is a classic computer science problem, the basis of diff (a file comparison program that outputs the differences between two files), and has applications in bioinformatics.
A subsequence is defined as an ordered subset of the array's elements having the same sequential ordering as the original array.
Given two sequences, find the length of longest subsequence present in both of them. For example,
For sequences "**ABAZDC**" and "**BACBAD**" the LCS is "**ABAD**" of length 4.
For sequences "**AGGTAB**" and "**GXTXAYB**" the LCS is "**GTAB**" of length 4.
Note: A string of length n has 2^n-1 different possible subsequences, which implies that the time complexity of the brute force approach will be O(n * 2^n). It takes O(n) time to check if a subsequence is common to both the strings. This time complexity can be improved using dynamic programming.
Input Format
The first line of input contains an integer T denoting the no of test cases. Then T test cases follow.
Each test case contains two strings in two lines.
Output Format
For each test case, print the length of the Longest Common Subsequence in new lines.
Sample Input
2
ABAZDC
BACBAD
AGGTAB
GXTXAYB
Sample Output
4
4

Solution:

```
import java.util.*;
class Result
{
   static int lcahelper(String s1,String s2,int n,int m,int dp[][]){
      if (n==0 || m==0){
         return 0;
      }
      if (dp[n][m]!=-1){
         return dp[n][m];
      }
      int mx=0;
      if (s1.charAt(n-1)==s2.charAt(m-1)){
         mx=1+lcahelper(s1,s2,n-1,m-1,dp);
      }else{
         int lans=lcahelper(s1,s2,n-1,m,dp);
         int rans=lcahelper(s1,s2,n,m-1,dp);
         mx = Math.max(lans,rans);
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        }
        dp[n][m]=mx;
        return mx;
    }
  static int longestCommonSubsequence(String str1, String str2){
    // Write your code here
      int[][] dp=new int[str1.length()+1][str2.length()+1];
      for (int i=0;i<=str1.length();i++){
           Arrays.fill(dp[i],-1);
      }
      return lcahelper(str1,str2,str1.length(),str2.length(),dp);
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: The Subset Sum problem

The subset sum problem is an important problem in computer science.
The challenge is to determine if there is some subset of numbers in an given array that can sum up to some given number S.
For example, if the array is {1, 4, 5, 10, 16} and sum = 9, you should return 1 and if sum = 7, you should return 0.

Input Format

First Line will contain an integer M denoting the sum.
Second Line will contain an integer N denoting the total number of elements.
Third line contains N integers separated by space.

Output Format

Print 1 if some subset found and 0 otherwise.

Sample Input

9
5
1 4 5 10 16

Sample Output

1

Solution:

```
class Result
{
   static boolean solve(int[] a,int n,int idx,int sum,int cnt){
      if (idx>n){
         return false;
      }
      if (sum==cnt){
         return true;
      }
      boolean f=solve(a,n,idx+1,sum,cnt+a[idx]);
      boolean s=solve(a,n,idx+1,sum,cnt);
      if (f || s) return true;
      return  false;
   }
  static int subsetSum(int a[], int n, int sum){
    // Write your code here
     int cnt=0;
     return solve(a,n,0,sum,cnt)?1:0;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Matrix Chain Multiplication problem

A matrix can be represented as a 2-dimensional array. Two matrices are eligible for multiplication if the number of columns in 1st matrix is equal to the number of rows in 2nd matrix.
The total number of operations required for multiplying two matrices A [m x n] and B [n x o] will be (m * n * o).
We have many ways to multiply a chain of matrices because matrix multiplication is associative, each way require different number of operations to complete. Mr. Amit has a sequence of matrices and interested to find the way in which minimum operations are required to multiply all matrices. The problem is not actually to perform the multiplications, but merely to decide in which order to perform the multiplications.
Example :
If we had four matrices A, B, C, and D, we would have:
(ABC)D or (AB)(CD) or A(BCD) and we can use the same kind of parenthesizes for inner problems to get ((A(BC))D) or (((AB)C)D) or (AB)(CD) or (A((BC)D)) or (A(B(CD))).
Suppose A is a 10 × 20 matrix, B is a 20 × 30 matrix, and C is a 30 × 40 matrix. Then,
For 3 matrices we have 2 options
(A(BC)) = To Multiply BC we need 20*30*40=24000 and we get 20x40 matrix, then it is multiplied with A and take 10*20*40=8000 operations, so total 32000 operations needed.
((AB)C) = To Multiply AB we need 10*20*30=6000 and we get 10x30 matrix, then it is multiplied with C and take 10*30*40=12000 operations, so total 18000 operations needed.
Input Format:
First Line will contain an integer N denoting the number of matrices.
Second line contains N+1 integers denoting the matrix sizes separated by space.
Output Format:
Print the minimum operations needed for matrix multiplication as specified above.
Constraints
2 <= N <= 100
1 <= arr[i] <= 500
Sample Input
3
10 20 30 40
Sample Output
18000

Solution:

```
import java.util.*;
class Result
{
  public static int  solve(int arr[],int i,int j,int dp[][]){
      if(i==j){
        return 0;
      }
      int ans=Integer.MAX_VALUE;
    for(int k=i;k<j;k++){
      int left=solve(arr,i,k,dp);
      int right=solve(arr,k+1,j,dp);
      int s3=arr[i-1]*arr[k]*arr[j];
      ans=Math.min(ans,left+right+s3);
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        }
        return ans;
    }
    static int matrixChainMultiplication(int arr[], int m){
    // Write your code here
        int dp[][] = new int[m + 1][m + 1];
        // Initialize the dp array
        for (int i = 1; i <= m; i++) {
            dp[i][i] = 0;
        }
        // Applying the dynamic programming approach
        for (int length = 2; length <= m; length++) {
            for (int i = 1; i <= m - length + 1; i++) {
                int j = i + length - 1;
                dp[i][j] = Integer.MAX_VALUE;
                for (int k = i; k < j; k++) {
                    int cost = dp[i][k] + dp[k + 1][j] + arr[i - 1] * arr[k] * arr[j];
                    dp[i][j] = Math.min(dp[i][j], cost);
                }
            }
        }
        return dp[1][m];
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: 0-1 Knapsack problem

The knapsack problem or rucksack problem is a problem in combinatorial optimization: Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.
In this problem we will solve a variant of it, in which the items can have a binary decision only i.e. either you can pick the item or leave it. No partial items allowed to fill the bag at fullest. This is also known as 0-1 knapsack problem.
Given two integer arrays values[0..n-1] and weight[0..n-1] which represent values and weights associated with n items respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of values[] such that sum of the weights of this subset is smaller than or equal to W.
Note: You cannot break an item, either pick the complete item, or don't pick it (0-1 property).
Input Format
First Line will contain an integer N denoting the number of items.
Second line contains N integers separated by space as values of N items.
Third line contains N integers separated by space as weights of N items.
Fourth line contains an integer denoting the capcacity of knapsack.
Output Format
Print the maximum value that can be earned with above knapsack.
Constraints
1 <= N <= 10^3
1 <= val[i] <= 10^3
1 <= weight[i] <= 10^3
1 <= capacity <= 10^3
Sample Input
3
60 100 120
10 20 30
50
Sample Output
220
Explanation
Pick 2nd and 3rd item with weight 20 and 30 and profit 100 and 120.

Solution:

```
import java.util.*;
class Result
{
   static int solve(int[] val,int[] wt,int cap,int idx,int dp[][]){
      if (idx==-1 || cap==0){
         return 0;
      }
      if (dp[idx][cap]!=-1){
         return dp[idx][cap];
      }
      int inc=0;
      if (wt[idx]<=cap){
         inc=val[idx]+solve(val,wt,cap-wt[idx],idx-1,dp);
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        }
        int exc=solve(val,wt,cap,idx-1,dp);
        dp[idx][cap]=Math.max(inc,exc);
        return dp[idx][cap];
    }
  static int zeroOneKnapsack(int val[], int weight[], int n, int capacity){
    // Write your code here
      int[][] dp=new int[n+1][capacity+1];
      for (int i=0;i<=n;i++){
          Arrays.fill(dp[i],-1);
      }
      return solve(val,weight,capacity,n-1,dp);
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Tom And Permutations

Tom loves to generate new strings by using various different methods. So, this time he decided to generate new strings by just rearranging the characters of some given string. In other words, he decided to find all the permutations for a given string and print them.
You task is to help Tom write a function to find all the permutations of a given string, that contains distinct characters.

Input Format:
First lines contains a string

Output Format:
Print all permutations of given string in lexicographical order

Sample Input
ABC

Sample Output
ABC
ACB
BAC
BCA
CAB
CBA

Solution:

```
class Solve {
  /*
   * Return all the permutations of the given string in any order,
   * as they will be sorted at the back-end before printing.
   */
    public static void solve(String str,String prmut,ArrayList<String> arr){
        if (str.length()==0){
            arr.add(prmut);
            return ;
        }
        for (int i=0;i<str.length();i++){
            char ch= str.charAt(i);
            String newstr= str.substring(0,i)+str.substring(i+1);
            solve(newstr,prmut+ch,arr);
        }
    }
  ArrayList<String> permute(String str) {
    // Write your code here
      ArrayList<String> arr=new ArrayList<>();
      solve(str,"",arr);
      return arr;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Print all strings of n-bits

Given n-bits we can generate 2^n different strings from it. For example, with 3 bits we can generate 2^3=8 strings i.e. 000, 001, 010, 011, 100, 101, 110, 111.
You need to write a function to find all strings for a given number of bits.
Input Format:
First lines contains an integer denoting the number of bits
Output Format:
Print all strings for given number of bits in ascending order.
Sample Input
3
Sample Output
000
001
010
011
100
101
110
111

Solution:

```
class Solve
{
    // The first argument is the number of bits. You need to save all binary strings in the ArrayList passed as 4th argument named strs.
   // Dont print the strings as they will be printed after needed processing (sorting in ascending order) at back end.
    // i is initially passed as 0, currStr is a char array to store the current String.
   void generateAllStrings(int n, int l, char currStr[], ArrayList<String> strs){
    // Write your code here
      for (int i=0;i<Math.pow(2,n);i++){
         int m=i;
         StringBuilder str=new StringBuilder();
         for (int j=0;j<n;j++){
            int rem=m%2;
            str.insert(0,rem);
            m/=2;
         }
         strs.add(str.toString());
      }
   }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Solve N Queen problem

The N Queen problem is the problem of placing N chess queens on an N×N chessboard so that no two queens attack each other.
You must know that a queen in chess can attack in all 8 directions.
Given the size of board as N*N, you need to count how many ways are there to place N queens on the board and save all such solutions in the given array

Input Format:
First lines contains an integer denoting the number of queens

Output Format:
Print all solutions in there column number for all queens.
If no solution exists, print -1.

Sample Input
4

Sample Output
1 3 0 2
2 0 3 1

Explanation:
Solutions are for per row starting from row 0 to row N-1.
1 3 0 2 means For 0th row pick column1, for 1st row pick column 3, for 2nd row pick column 0 and for 3rd row pick column 2.

There are total two solutions:
First is 1 3 0 2, means 1st queen at column 1, 2nd queen at column 3, 3rd queen at column 0 and 4th queen at column 2
Second is 2 0 3 1, means 1st queen at column 2, 2nd queen at column 0, 3rd queen at column 3 and 4th queen at column 1

Solution:

```
class Result
{
// Complete this function to check placing queen at board[row][col] is safe or not by checking current row, left diagonal & right diagonal.
  boolean isSafe(int board[][], int row, int col, int N)
  {
      // row
      for (int i=0;i<N;i++){
          if (board[row][i]==1){
              return false;
          }
      }
      // col
      for (int i=0;i<N;i++){
          if (board[i][col]==1){
              return false;
          }
      }
      // top left
      int r=row-1;
      for (int i=col-1;i>=0 && r>=0;i--,r--){
          if (board[r][i]==1){
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
                return false;
            }
        }
        // top right;
        r= row-1;
        for (int i=col+1;i<N && r>=0;i++,r--){
            if (board[r][i]==1){
                return false;
            }
        }
        // botm left
        r=row+1;
        for (int i=col-1;i>=0 && r<N;i--,r++){
            if (board[r][i]==1){
                return false;
            }
        }
        // botm right
        r=row+1;
        for (int i=col+1;i<N && r<N;i++,r++){
            if (board[r][i]==1){
                return false;
            }
        }
        return true;
    }
     void saveBoard(int[][] board,ArrayList<ArrayList<Integer>> sol,int N){
         ArrayList<Integer> boardList = new ArrayList<>();
         for (int i=0;i<N;i++){
             for(int j=0;j<N;j++){
                 if (board[i][j]==1){
                     boardList.add(j);
                 }
             }
         }
         sol.add(boardList);
     }
    // Complete this function to solve the problem and save the answers in sol ArrayList as
required.
    boolean solveNQUtil(int board[][], int col, int N, ArrayList<ArrayList<Integer> > sol)
    {
        // System.err.println(N);
        if (col==N){
            saveBoard(board,sol,N);
            return true;
        }
        boolean res= false;
        for (int i=0;i<N;i++){
            if (isSafe(board,i,col,N)){
                board[i][col]=1;
                res=solveNQUtil(board,col+1,N,sol) || res;
                board[i][col]=0;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
            }
        }
        return res;
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Solve Sudoku

Sudoku is one of the most popular puzzle games of all time. The goal of Sudoku is to fill a 9×9 grid with numbers so that each row, column and 3×3 section contain all of the digits between 1 and 9. As a logic puzzle, Sudoku is also an excellent brain game. If you play Sudoku daily, you will soon start to see improvements in your concentration and overall brain power.
So, given a partially filled 9×9 matrix i.e. mat[9][9], Your goal is to assign digits (from 1 to 9) to the empty cells so that every row, column, and sub-grid of size 3×3 contains exactly one instance of the digits from 1 to 9.
If the given sudoku is not solvable, or having wrong prefilled entries, you should return -1, otherwise on successful solution return 1.

Input Format

9 lines contains 9 integers (0-9) separated by space as elements of the sudoku matrix.
Where 0 indicates null entry and other numbers are prefilled entries.

Output Format

Print the solved sudoku as shown.

Sample Input

```
2 9 0 8 7 3 0 1 0
4 0 0 0 0 5 9 2 0
0 1 0 0 2 4 0 0 0
0 0 0 0 8 9 6 0 0
0 0 4 0 0 0 8 3 0
0 8 2 3 1 0 5 0 0
0 0 9 2 3 8 0 0 7
8 0 0 0 4 7 0 0 0
3 0 5 0 9 0 2 8 4
```

Sample Output

```
2 9 6 8 7 3 4 1 5
4 3 7 1 6 5 9 2 8
5 1 8 9 2 4 7 6 3
1 5 3 4 8 9 6 7 2
9 6 4 7 5 2 8 3 1
7 8 2 3 1 6 5 4 9
6 4 9 2 3 8 1 5 7
8 2 1 5 4 7 3 9 6
3 7 5 6 9 1 2 8 4
```

Solution:

```
class Result {
    static boolean isSafe(int[][] mat,int row,int col,int n){
        for (int i=0;i<mat.length;i++){
            if (mat[row][i]==n){
                return false;
            }
            if (mat[i][col]==n){
                return false;
            }
        }
        int sr= (row/3)*3;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        int sc= (col/3)*3;
        for (int i=sr;i<sr+3;i++){
            for(int j=sc;j<sc+3;j++){
                if (mat[i][j]==n)
                    return false;
            }
        }
        return true;
    }
    static boolean helper(int[][] mat,int row,int col){
        if (row==mat.length){
            return true;
        }
        int nrow=0;
        int ncol=0;
        if (col!=mat.length-1){
            nrow=row;
            ncol=col+1;
        }else{
            nrow=row+1;
            ncol=0;
        }
        if (mat[row][col]!=0){
            if (helper(mat,nrow,ncol))
                return true;
        }else{
            for (int i=1;i<=9;i++){
                if (isSafe(mat,row,col,i)){
                    mat[row][col]=i;
                    if (helper(mat,nrow,ncol)){
                        return true;
                    }else{
                        mat[row][col]=0;
                        //return false;
                    }
                }
            }
        }
        return false;
    }
  static int solveSudoku(int mat[][])
  {
     return helper(mat,0,0) ? 1:-1;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Rat in a Maze Problem

A Maze is given as N*N binary matrix of blocks. A rat starts from **source** and has to reach the **destination**.
The rat can move only in two directions, i.e. **rightwards** and **downwards**.
In the maze matrix, -1 means the block is a dead end and 0 means the block can be used in the path from **source** to **destination**.
Note: Source block is the upper left most block i.e., maze[0][0] and **destination** block is lower rightmost block i.e., maze[N-1][N-1]
Your task is to find the number of possible solutions for the rat to reach the destination.
Input Format
First line contains number of rows i.e. N
Next N lines contains N binary numbers each denoting the maze
Constraints
2 <= N <= 15
Output Format
Print the number of solutions
Sample Input
4
0 -1 -1 -1
0 0 -1 -1
-1 0 0 -1
-1 0 0 0
Sample Output
2
Explanation:
Solution 1: From cell (0,0) -> down -> (1,0) -> forward -> (1,1) -> down -> (2,1) -> down -> (3,1) -> forward -> (3,2) -> forward -> (3,3)
Solution 2: From cell (0,0) -> down -> (1,0) -> forward -> (1,1) -> down -> (2,1) -> forward -> (2,2) -> down -> (3,2) -> forward -> (3,3)
Total 2 solutions possible for this maze.

Solution:

```
import java.util.*;
class Result {
   /*public static boolean isSafe(int[][] maze,int x,int y,int[][] vist){
      return (x>=0 && x<maze.length && y>=0 && y<maze[0].length && vist[x][y]==0
&& maze[x][y]==0);
   }
   public static int helper(int[][] maze,int x,int y,int[][] vist,int size){
      if (x==size-1 && y==size-1){
         return 1;
      }
      vist[x][y]=1;
      int cnt=0;
      // right
      int sr=x,sc=y+1;
      if (isSafe(maze,sr,sc,vist)){
         cnt+=helper(maze,sr,sc,vist,size);
      }
      // down
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        sr=x+1;
        sc=y;
        if (isSafe(maze,sr,sc,vist)){
            cnt+=helper(maze,sr,sc,vist,size);
        }
        vist[x][y]=0;
        return cnt;
    }
  public static int solveMaze(int maze[][], int size) {
    // Write your code here
      if (maze[0][0]==-1){
          return 0;
      }
      int[][] vist=new int[size][size];
      for (int i=0;i<size;i++){
          Arrays.fill(vist[i],0);
      }
      return helper(maze,0,0,vist,size);
  }
    */
   public static boolean isSafe(int[][] maze, int x, int y, int[][] vist,int size) {
        return (x >= 0 && x < size && y >= 0 && y < size && vist[x][y] == 0 && maze[x][y]
== 0);
    }
    public static int helper(int[][] maze, int x, int y, int[][] vist, int size) {
        if (x == size - 1 && y == size - 1) {
            return 1;
        }
        vist[x][y] = 1;
        int[] rowNum = { 1, 0 }; // Down, Right
        int[] colNum = { 0, 1 }; // Down, Right
        int count = 0;
        for (int i = 0; i < 2; i++) {
            int sr = x + rowNum[i];
            int sc = y + colNum[i];
            if (isSafe(maze, sr, sc, vist,size)) {
                count += helper(maze, sr, sc, vist, size);
            }
        }
        vist[x][y] = 0; // Backtrack
        return count;
    }
    public static int solveMaze(int maze[][], int size) {
        if (maze[0][0] == -1) {
            return 0;
        }
        int[][] vist = new int[size][size];
        for (int i = 0; i < size; i++) {
            Arrays.fill(vist[i], 0);
        }
        return helper(maze, 0, 0, vist, size);
    }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Count word in the board

Problem

Given a **m** x **n** 2D board of characters and a word, find how many times the word is present in the board.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring.

Note: The same letter cell may not be used more than once.

Input Format

First line contains two space separated integers m, n that denotes the number of rows and columns in the board respectively.

Next m lines contain n characters each.

Last line contains the word to be searched.

Constraints

1 <= m, n <= 5

1 <= lengthOf(word) <= 30

Output Format

Print the total count of the given word in the 2-D board.

Sample Input 1

```
4 4
COXG
ADTA
BERM
ACDE
CODER  // word
```

Sample Output 1

```
1
```

Sample Input 2

```
3 4
HDST
ANEO
BENY
NEST  // word
```

Sample Output 2

```
2
```

Solution:

```
class Result {
   static boolean isSafe(char board[][], boolean[][] vist, String word, int idx, int m, int n, int
x, int y) {
       return (x >= 0 && x < m && y >= 0 && y < n && idx < word.length() && board[x][y]
== word.charAt(idx)
               && !vist[x][y]);
   }
   static int dfs(char[][] board, boolean[][] vist, String word, int idx, int r, int c, int m, int n) {
       if (idx == word.length() - 1) {
           return 1;
       }
       int cnt = 0;
       int[] row = { -1, 0, 1, 0 };
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        int[] col = { 0, 1, 0, -1 };
        vist[r][c] = true;
        for (int i = 0; i < 4; i++) {
            int nrow = r + row[i];
            int ncol = c + col[i];
            if (isSafe(board, vist, word, idx + 1, m, n, nrow, ncol)) {
                cnt += dfs(board, vist, word, idx + 1, nrow, ncol, m, n);
            }
        }
        vist[r][c] = false;
        return cnt;
    }
    static int countWord(char board[][], String word, int m, int n) {
        // Write your code here
        int cnt = 0;
        boolean[][] vist = new boolean[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == word.charAt(0)) {
                    cnt += dfs(board, vist, word, 0, i, j, m, n);
                }
            }
        }
        return cnt;
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find the minimum number of edges in a path of a graph

Consider a directed graph whose vertices are numbered from 1 to n. There is an edge from a vertex i to a vertex j iff either j = i + 1 or j = 3i. The task is to find the minimum number of edges in a path in G from vertex 1 to vertex n.

Input Format:

The first line of input contains an integer T denoting the number of test cases. The description of T test cases follows.

Each test case contains a single line of input which contains an integer, n.

Output Format:

Print the number of edges in the shortest path from 1 to n.

Sample Input

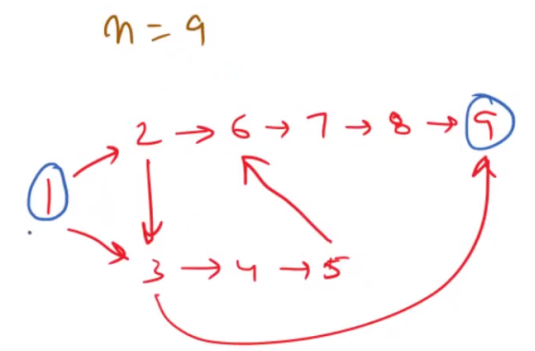
1
9

Sample Output

2

Explanation:

The below given graph is formed



The minimum number of edges from vertex 1 to vertex 9 is 2.

Solution:

```
class Result{
  static int number_of_edges(int n){
    // Write your code here
      Queue<Integer> q=new LinkedList<>();
      boolean[] vis=new boolean[n+1];
      int[] dis=new int[n+1];
      q.add(1);
      while(!q.isEmpty()){
          int curr = q.poll();
          int[] nexts={1+curr,3*curr};
          for (int next:nexts){
              if (next<=n && !vis[next]){
                  vis[next]=true;
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
                dis[next]=dis[curr]+1;
                q.add(next);
                if (next==n){
                    return dis[next];
                }
            }
        }
    }
    return -1;
  }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find path in a directed graph

Given a **directed graph** and two vertices(say source and destination vertex), check whether there exists some path from the given source vertex to the destination vertex or not. If it exists then print “YES”, else print “NO”.

Input Format:

First line contains two space separated integers V, E denoting the number of vertices and edges in the graph.

Following E lines contain two space separated integers u, v denoting a directed edge from u to v.

Last line contains two space separated integers src, dest denoting the source and destination vertex respectively.

Constraints:

1 <= V <= 100

0 <= E < 10000

0 <= u, v, src, dest < 100

Output Format:

Print “YES” if a path exists from the given source vertex to the destination vertex, else print “NO”.

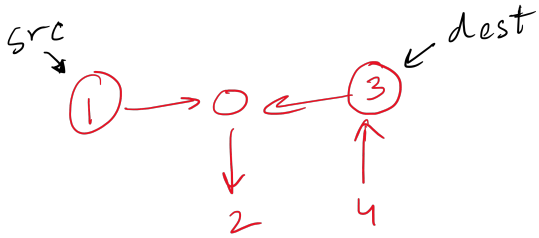
Sample Input 1

```
5 4  // V E
1 0  // directed edge from u to v
0 2
3 0
4 3
1 3  // source destination
```

Sample Output 1

NO

Explanation 1



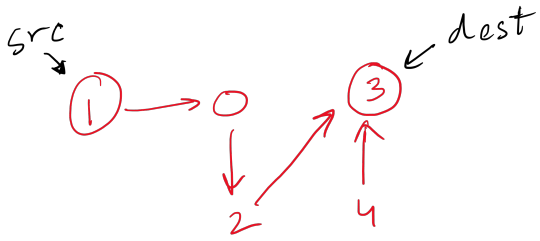
No path exists, from the given source vertex to the destination vertex

Sample Input 2

```
5 4  // V E
1 0
0 2
2 3
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

4 3
1 3
Sample Output 2
YES
Explanation 2



There is a path, from the given source vertex to the destination vertex

Solution:

```
import java.util.Scanner;
// Other imports go here
// Do NOT change the class name
import java.util.*;
class gh{
    int v;
    Map<Integer,List<Integer>> adj;
    public gh(int v){
        this.v=v;
        this.adj=new HashMap<Integer,List<Integer>>();
    }
    public void addvertx(int i){
        adj.putIfAbsent(i,new ArrayList<Integer>());
    }
    public void addedge(int s,int d){
        adj.get(s).add(d);
    }
//}
//class solution{
    public boolean ispath(boolean[] vis,int s,int d){
        if(s==d){
            return true;
        }
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        vis[s]=true;
        for (int i=0;i<adj.get(s).size();i++){
            int adjnode= adj.get(s).get(i);
            if (!vis[adjnode] && ispath(vis,adjnode,d)){
                return true;
            }
        }
        return false;
    }
}
class Main{
    public static void main(String[] args)
    {
        // Write your code here
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int e=sc.nextInt();
        gh g=new gh(n);
        for (int i=0;i<n;i++){
            g.addvertx(i);
        }
        for (int i=0;i<e;i++){
            int s=sc.nextInt();
            int d=sc.nextInt();
            g.addedge(s,d);
        }
        int s=sc.nextInt();
        int d=sc.nextInt();
        //solution sb=new solution();
        boolean[] vis=new boolean[n+1];
        System.out.print(g.ispath(vis,s,d) ? "YES":"NO");
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Number of Islands

Given a **m** x **n** binary matrix in which 1 represents 'land' and 0 represents 'water', find the number of islands.
An island is surrounded by water(**0s**) and is formed by connecting adjacent lands(**1s**) horizontally or vertically. You may assume all four edges of the given matrix are all surrounded by water.

Input Format:
First line contains two space separated integers m, n that denotes the number of rows and columns in the matrix respectively.
Each of the next m lines contain n space-separated integers representing the matrix.

Constraints:
1 <= m, n <= 150

Output Format:
Print the number of islands.

Sample Input
4 4
1 0 1 0
1 1 0 0
0 0 0 1
1 1 1 1

Sample Output
3

Explanation:
First island consists of following cells : (0, 0), (1, 0), (1,1)
Second island consists of following cells : (0, 2)
Third island consists of following cells : (2, 3), (3, 0), (3, 1), (3, 2), (3, 3)

Solution:

```
class Result {
   static boolean isSafe(int[][] mat,int x,int y,int m,int n){
      return (x>=0 && x<m && y>=0 && y<n && mat[x][y]==1);
   }
   static void solver(int[][] mat,int x,int y){
      if(!isSafe(mat,x,y,mat.length,mat[0].length)){
         return ;
      }
      mat[x][y]=-1;
      solver(mat,x-1,y);
      solver(mat,x+1,y);
      solver(mat,x,y-1);
      solver(mat,x,y+1);
   }
 static int countIslands(int mat[][], int m, int n){
   // Write your code here
     int cnt=0;
     for (int i=0;i<m;i++){
        for (int j=0;j<n;j++){
           if (mat[i][j]==1){
              cnt++;
              solver(mat,i,j);
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
                }
            }
        }
        return cnt;
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Shortest path in a binary maze

Given a **m** x **n** binary matrix, find the length of the shortest path in the matrix from a given source cell to a destination cell. The path can only be created out of a cell if its value is 1. If there is no path from a given source cell to a destination cell, return -1.
Note: At any point of time, you can either move horizontally or vertically in the four directions.
Input Format:
First line contains two space separated integers m, n that denotes the number of rows and columns in the matrix respectively.
Each of the next m lines contain n space-separated integers representing the matrix.
Second Last line contains two space separated integers, denoting the position of the source cell.
Last line contains two space separated integers, denoting the position of the destination cell.
Constraints:
1 <= m, n <= 150
Output Format:
Print the length of the shortest path from source to destination.
Sample Input
5 5
0 1 1 1 1
1 1 0 0 1
1 0 0 1 1
1 1 1 1 0
0 1 0 1 0
1 1
3 2
Sample Output
5
Explanation:
The shortest path from (1, 1) to (3, 2) is as follows:
(1, 1) -> (1, 0) -> (2, 0) -> (3, 0) -> (3, 1) -> (3, 2)

Solution:

```
class Pair{
   int fst;
   int sec;
   int dist;
   public Pair(int f,int s,int d){
      this.fst=f;
      this.sec=s;
      this.dist=d;
   }
}
class Result {
   static int shortestPath(int mat[][], int srcR, int srcC, int destR, int destC, int m, int n){
     // Write your code here
      if (mat[srcR][srcC]==0 || mat[destR][destC]==0) return -1;
      if (srcR==destR && srcC==destC) return 0;
      Queue<Pair> q=new LinkedList<Pair>();
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        int[][] dist=new int[m][n];
        for (int i=0;i<m;i++){
            Arrays.fill(dist[i],Integer.MAX_VALUE);
        }
        q.add(new Pair(srcR,srcC,0));
        while(!q.isEmpty()){
            Pair temp=q.poll();
            int f=temp.fst;
            int s=temp.sec;
            int d=temp.dist;
            int[] row={1,0,-1,0};
            int[] col={0,1,0,-1};
            for (int i=0;i<4;i++){
                int nrow=f+row[i];
                int ncol=s+col[i];
                if (nrow>=0 && nrow<m && ncol>=0 && ncol<n && mat[nrow][ncol]==1 &&
d+1<dist[nrow][ncol]){
                    dist[nrow][ncol]=d+1;
                    if (nrow==destR && ncol==destC){
                        return d+1;
                    }
                    q.add(new Pair(nrow,ncol,d+1));
                }
            }
        }
        return -1;
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Depth First Traversal of Graph

Graphs can be traversed popularly in following two ways: Depth First Traversal (DFT) and Breadth First Traversal (BFT). Traversing Graphs can be little bit more tricky than traversing tree because of possibilities of a cycle in graph. So, you need to to write the function for doing a depth first traversal over the given graph.

Input Format:

First line contains an integer V i.e. number of nodes.
Second line contains an integer E i.e. number of edges.
Next E lines contains two integers each v1, v2 denoting the edge from v1 to v2.
Last line of input contains an integer denoting the starting vertex for DFT.

Output Format:

Print the vertex labels starting from given vertex separated by space.

Sample Input

```
4
5
0 1
0 2
1 2
2 0
2 3
2
```

Sample Output

```
2 0 1 3
```

Solution:

```
/* The class is defined with below variables
  class Graph
{
  private Map<Integer, List<Integer>> adjVertices;
  public Graph() {
    this.adjVertices = new HashMap<Integer, List<Integer>>();
  }
  public void addVertex(int vertex) {
    adjVertices.putIfAbsent(vertex, new ArrayList<>());
  }
  public void addEdge(int src, int dest) {
    adjVertices.get(src).add(dest);
  }       */
void DFSUtil(int v, boolean vis[])
{
    System.out.print(v+" ");
    vis[v]=true;
    for (int i=0;i<adjVertices.get(v).size();i++){
        int adjnode= adjVertices.get(v).get(i);
        if (!vis[adjnode]){
            DFSUtil(adjnode,vis);
        }
    }
}
void DFS(int v)
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
{
   int V=adjVertices.size();
   boolean[] vis=new boolean[V];
   DFSUtil(v,vis);
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Breadth First Traversal of Graph

Graphs can be traversed popularly in following two ways: Depth First Traversal (DFT) and Breadth First Traversal (BFT). Traversing Graphs can be little bit more tricky than traversing tree because of possibilities of a cycle in graph. So, you need to to write the function for doing a breadth first traversal over the given graph.

Input Format:

First line contains an integer V i.e. number of nodes.
Second line contains an integer E i.e. number of edges.
Next E lines contains two integers each v1, v2 denoting the edge from v1 to v2.
Last line of input contains an integer denoting the starting vertex for BFT.

Output Format:

Print the vertex labels starting from given vertex separated by space.

Sample Input

```
4
5
0 1
0 2
1 2
2 0
2 3
2
```

Sample Output

```
2 0 3 1
```

Solution:

```
/* The class is defined with below variables
class Graph {
  private int V;
  private Map<Integer, List<Integer>> adjVertices;
  public Graph(int V) {
      this.V = V;
    this.adjVertices = new HashMap<Integer, List<Integer>>();
  }
  public void addVertex(int vertex) {
    adjVertices.putIfAbsent(vertex, new ArrayList<>());
  }
  public void addEdge(int src, int dest) {
    adjVertices.get(src).add(dest);
  }       */
void BFS(int v)
{
    Queue<Integer> q=new LinkedList<>();
    boolean[] vist= new boolean[V];
    vist[v]=true;
    q.add(v);
    while(!q.isEmpty()){
        int top=q.remove();
        System.out.print(top+" ");
        for (int i=0;i<adjVertices.get(top).size();i++){
            int adjnode=adjVertices.get(top).get(i);
```

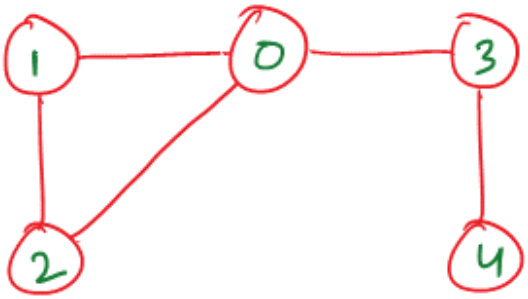

SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
            if (!vist[adjnode]){
                vist[adjnode]=true;
                q.add(adjnode);
            }
        }
    }
}
```

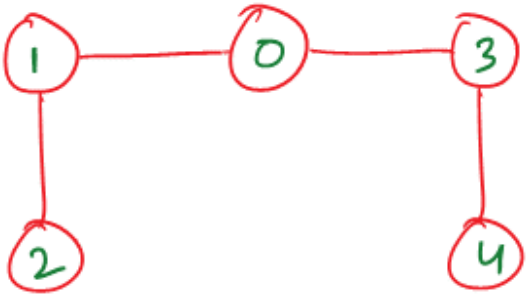

SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

Aim: Find the cycle in undirected graph

Given a un-directed graph find out if there is a cycle in it or not.
Note: The given graph **may** or **may not** be connected.
Example 1: The following graph contains a cycle



Example 2: The following graph does **not** contain a cycle



Input Format
The first line contains the Number of vertices, V.
The second line contains the Number of edges, E.
Then E lines follow , containing 2 numbers,x & y, seperated by space denoting an edge from vertex x to y.
Output Format
Print Yes or No depending on graph has cycle or not.
Sample Input

```
5  // Vertices
5  // Edges
0 1  // Edge from v1 to v2
0 2
1 2
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

0 3
3 4

Sample Output

Yes

Explanation:

The graph is having a cycle from vertices 0, 1 and 2

Solution:

```
import java.util.*;
class CreateGraph {
   public Map<Integer, List<Integer>> adj;
   public CreateGraph() {
      this.adj = new HashMap<>();
   }
   public void addVertex(int v) {
      adj.putIfAbsent(v, new ArrayList<>());
   }
   public void graphCreation(int src, int dest) {
      adj.get(src).add(dest);
      adj.get(dest).add(src); // Adding bidirectional edges for undirected graph
   }
}
class Solution extends CreateGraph {
   public void printgh() {
      for (Map.Entry<Integer, List<Integer>> entry : adj.entrySet()) {
         System.out.print(entry.getKey() + " -> ");
         for (Integer i : entry.getValue()) {
            System.out.print(i + " ");
         }
         System.out.println();
      }
   }
   public boolean isCycleUndirected(boolean[] vis, int curr, int par) {
      vis[curr]=true;
      for (int i=0;i<adj.get(curr).size();i++){
         int eg= adj.get(curr).get(i);
         if(!vis[eg] ){
            if (isCycleUndirected(vis,eg,curr)){
            return true;
            }
         }
         else if (eg!=par){
            return true;
         }
      }
      return false;
   }
}
public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in

```
        int v = sc.nextInt();
        int e = sc.nextInt();
        CreateGraph cg = new CreateGraph();
        for (int i = 0; i < v; i++) {
            cg.addVertex(i);
        }
        for (int i = 0; i < e; i++) {
            int src = sc.nextInt();
            int dest = sc.nextInt();
            cg.graphCreation(src, dest);
        }
        Solution sb = new Solution();
        sb.adj = cg.adj;
        boolean[] vis = new boolean[v];
        boolean hasCycle = false;
        for (int i = 0; i < v; i++) {
            if (!vis[i]) {
                if (sb.isCycleUndirected(vis, i, -1)) {
                    hasCycle = true;
                    break;
                }
            }
        }
        //sb.printgh();
        if (hasCycle) {
            System.out.println("Yes");
        } else {
            System.out.println("No");
        }
    }
}
```


SAKSHAM GARG
saksham2221.be22@chitkara.edu.in